



UNIVERSITÀ DI PARMA

DIPARTIMENTO DI INGEGNERIA E ARCHITETTURA

Corso di Laurea Magistrale in Ingegneria Informatica

Applicazione di Reinforcement Learning a Spazi di Stato Ampi e Parzialmente Osservabili in un Ambiente Videoludico

*Application of Reinforcement Learning to Large, Partially
Observable State Spaces in a Video Game Environment*

CANDIDATO:
Andrea Bersellini

RELATORE:
Prof. Andrea Prati

CORRELATORI:
Prof.ssa Ana Maria de Almeida
Prof. Luís Nunes

Contents

Introduction	1
1 Literature Review	5
1.1 Objectives	5
1.2 PRISMA	5
1.3 Methods	6
1.3.1 Eligibility Criteria	6
1.3.2 Information Sources	6
1.3.3 Selection Process	7
1.4 Findings	7
1.5 Discussion	9
2 Preliminaries	11
2.1 Reinforcement Learning	11
2.1.1 Q-Learning	13
2.1.2 Deep Q-Learning	14
2.2 State-Space Trade-Off	16
2.3 Exploration vs. Exploitation	17
2.4 Tools and Libraries	18
2.4.1 G2D Library	18
3 Implementation	19
3.1 Environment	19
3.1.1 Actors	20
3.1.2 Game Mechanics	21
3.1.3 Graphics	21
3.2 Exploration Policy	22
3.3 Reward Function	24
3.4 State-Action Memory	26
3.4.1 Experience Replay	28
3.5 Baseline Q-Learning	28

3.5.1	State Space	29
3.6	Perceptive Q-Learning	30
3.6.1	State Space	30
3.7	Perceptive Deep Q-Learning	32
3.7.1	State Space	34
3.8	Visual Deep Q-Learning	34
3.8.1	State Space	36
4	Experimental Results	39
4.1	Evaluation Methodologies	39
4.1.1	Early Stopping	41
4.2	Training Outcomes	42
	Conclusions	53
	Bibliography	55

List of Figures

2.1	Interaction of an agent with the environment.	12
2.2	Example of a simple Markov Decision Process.	12
3.1	Screenshot of the environment taken during gameplay.	19
3.2	Representation of the abstract class <i>Actor</i>	21
3.3	Adaptive ϵ -greedy behaviour based on distance.	23
3.4	Example showing the use of the compact Q-table in the training cycle.	27
3.5	Environmental attributes composing the observable state.	31
3.6	Comparison between ReLu (on left) and Tanh (on right) activation functions.	33
3.7	Simplified python implementation of the network training cycle.	34
3.8	Visualization of PatchLayer operations.	35
3.9	Three consecutive frames stacked along the channel dimension generating the state.	37
4.1	Python implementation of the early stopping mechanism.	42
4.2	Performance and reward trend of the baseline agent during training.	44
4.3	Performance and reward trend of the baseline agent in the test runs.	45
4.5	Performance and reward trend of the Perceptive Q-Learning agent during training.	46
4.6	Performance and reward trend of the Perceptive Q-Learning agent in the test runs.	47
4.8	Performance and reward trend of the Perceptive Deep Q-Learning agent during training.	48
4.9	Performance and reward trend of the Perceptive Deep Q-Learning agent in the test runs.	48
4.10	Perceptive Deep Q-Learning training loss over time.	49
4.11	Performance and reward trend of the Visual Deep Q-Learning agent during training.	50
4.12	Performance and reward trend of the Visual Deep Q-Learning agent in the test runs.	50

4.13	Example feature map produced by the PatchLayer of the trained network.	51
4.14	Example feature map produced by the convolutional layer of the trained network.	52

List of Algorithms

1	Epsilon Value Computation	23
2	Reward Function	25

List of Tables

1.1	Eligibility criteria for identifying studies.	6
1.2	Included studies after the screening.	8
3.1	Reward values for each experiment.	26
3.2	Perceptive Deep Q-Network architecture.	32
3.3	Visual Deep Q-Network architecture.	36
4.1	Q-Learning Training Hyper-parameters.	40
4.2	Deep Q-Learning Training Hyper-parameters.	40
4.3	Training iterations for every experiment.	40
4.4	Test results of each model after training.	43
4.5	Train statistics of each approach. *Preemptive early-stopping	43

List of Acronyms

CNN Convolutional Neural Network.

DNN Deep Neural Network.

DQL Deep Q-Learning.

DQN Deep Q-Network.

DRL Deep Reinforcement Learning.

MLP Multi Layer Perceptron.

PRISMA Preferred Reporting Items for Systematic reviews and Meta-Analyses.

QL Q-Learning.

RL Reinforcement Learning.

SLR Systematic Literature Review.

Thanks

First of all, I would like to thank my supervisor, Prof. Andrea Prati, for his interest in this thesis work and for providing valuable advice during the course of this project.

I would also like to thank my co-supervisors, Prof. Ana Maria de Almeida and Prof. Luís Nunes, who welcomed me during my Erasmus period at their university and followed the progression of the stages of the project's development. Their support was fundamental to the success of this thesis project and to the welcoming experience I had at the host institution abroad.

In addition, I would like to express my sincere gratitude to all the professors who, by carrying out their work with dedication and professionalism, enabled me to build my knowledge in the field of computer engineering during these two years of my Master's degree.

In particular, I would like to thank Prof. Michele Tomaiuolo for providing the implementation of the graphics library on which part of this work is based.

My deepest gratitude goes to my parents, who have always supported my choices and helped me achieve them, even by making great sacrifices, and to my grandparents, who have always believed in me and in my abilities.

Finally, I would like to thank all my friends who have supported me and made the experiences truly special during this university journey.

Thank you all.

Introduction

Among the many machine learning techniques, Reinforcement Learning (RL) has become increasingly popular in the field of computer science due to its versatility across a wide range of applications and its similarity to the way humans learn to perform tasks. Thanks to significant advancements in deep learning and computational power, RL has evolved, over the course of these years, with new approaches and technologies, reaching the point where it can solve complex problems involving large state spaces.

In many everyday applications, trial-and-error learning techniques such as RL, enable various ways to develop "smart" systems that are more adaptable and prepared to unexpected scenarios, unlike the standard supervised methods where the data should be known a priori. Furthermore, RL can be tuned and controlled in how it learns, making systems more predictable and transparent in their decision-making. As with any other method, RL also presents several disadvantages. High-dimensional state-space problems, limited knowledge of important aspects of the environment, and the complexity of reward engineering make these models hard to train and often difficult to implement.

RL is often applied in domains where the complexity of the problem exceeds the capabilities of traditional algorithms and where the environment is inherently interactive, allowing an agent to explore and act. In robotics, RL is used to enable robots to navigate their surroundings, avoid obstacles, and detect or manipulate objects. In autonomous driving, the use of RL helps automate decision-making in complex environments: through trial and error, agents learn the consequences of their actions and develop action policies that are less biased by human misconceptions. RL is also employed in the financial sector, where "intelligent" systems can learn to predict market trends and make decisions based on interactions with a constantly evolving financial environment. Perhaps one of the most renowned and successful applications of RL is in game-playing systems, where algorithms such as AlphaGo [1] and AlphaZero [2] have defeated professional players in their respective games. Beyond entertainment, these models provide controlled and risk-free environments in which systems can be tested before being adapted to more demanding real-world applications.

Although RL is not yet widely used in everyday applications, it has the potential to be integrated into common devices to enhance their adaptability and performance. For this reason, many private companies and academic figures are actively pushing researches in this field. Motivated by this potential, Prof. Ana Maria de Almeida and Prof. Luís Nunes from ISCTE University Institute of Lisbon, where part of this research was conducted during a six months mobility period, supported the development of this thesis, contributing to the exploration of RL applied to high-dimensional state spaces and image-sequence problems and to the assessment of its benefits and limitations.

This study scientifically compares four RL approaches based on Q-Learning (QL) and Deep Q-Learning (DQL) within the same custom environment, evaluating their overall ease of use and performance. All experiments reported in this study are conducted using a custom-built video game as the environment, which allows the complexity of the problem to be adjusted and provides full control over all variables. Each approach is explained in detail, supported by selected portions of the source code, and evaluated independently of the effectiveness of its final results. In particular, for approaches that fail, the underlying issues are examined to identify potential causes.

Starting with the use of the QL technique, the first approach is conducted by using the absolute position of the player in the environment and storing, at every step, a Q-value corresponding to the current state-action pair in a pre-initialized Q-table. The second experiment uses a state representation based on the relative position of the player with respect to surrounding objects in the environment. This technique imitates the behaviour of an agent equipped with sensors that can perceive a limited number of elements in its vicinity. The third test is conducted using a DQL approach, in which the Q-table is replaced with a neural network. In this case, the agent creates a policy based on predictions rather than precomputed values, which allows it to act in previously unseen states and to form a more abstract policy. The model is evaluated using the same environment observations as in the previous test, but the rewards are adjusted to facilitate the training of the network. Finally, the last experiment combines the DQL approach with the use of an image-based representation taken directly from the environment at each step, allowing the agent to perceive everything a human player would see while playing the game.

Each chapter of this thesis is intended to help the reader understand the fundamental concepts and the state of the art of RL relevant to this study, comprehend the implementation process with its challenges and proposed solutions, and analyse the findings achieved during the research. To provide a clearer overview, the structure of the thesis is organized as follows:

- Chapter 1 explains the documentation process used to identify relevant lit-

erature on the subject topics and justifies the motivation of the project in relation to the current state of the art.

- Chapter 2 introduces the key concepts of RL and its subcategories, presents the main challenges of each approach, lists the tools and libraries used in the implementation, and defines the terminology adopted throughout the thesis.
- Chapter 3 describes the implementation of the environment, the private and shared modules used across the different approaches, and the parameters employed during model training. This chapter also discusses the problems encountered during development and the corresponding solutions proposed.
- Chapter 4 details the evaluation methodologies adopted during testing, along with the training hyperparameters and performance metrics. It presents the experimental outcomes, focusing on both training and validation performance.
- Lastly, the Conclusions section presents a summary of the work, discusses the main findings, and outlines directions for future research.

Chapter 1

Literature Review

1.1 Objectives

The purpose of this literature review is to explore the current state of Reinforcement Learning (RL) and Deep Reinforcement Learning (DRL) applied to partially observable image-sequence-based problems, particularly in the context of video-games, and to identify popular approaches to this area of study, as well as to evaluate both achievements and limitations.

The first objective is to determine whether using DRL with raw-image input is a feasible solution for certain types of image-sequence-based problems.

Another key task, given the high complexity of image-sequence-based problems, is to investigate whatever optimization techniques have been employed to reduce computational costs and improve efficiency, ideally allowing for real-time applications.

Lastly, to help guide the research and early development, it is important to identify a baseline architecture that could serve as a solid starting point for more complex implementations.

1.2 PRISMA

An essential measure to ensure validity in a research project is to maintain transparency in how the Systematic Literature Review (SLR) is conducted and to clearly justify the criteria for including or excluding sources. To standardize the reporting of SLRs, Preferred Reporting Items for Systematic reviews and Meta-Analyses (PRISMA) [3] provides a structured set of guidelines, supported by an abstract checklist, that ensures clarity and reproducibility in the documentation process. Each stage of the review can be systematically documented, including the motivations for conducting the review, the methods applied to select studies, and the

results obtained following the screening process.

The selection procedure is divided into three principal phases. First, the identification phase involves retrieving records from the selected database sources. Second, during the screening phase, the retrieved records are assessed against predefined eligibility criteria, and those that do not meet the requirements are excluded. Finally, the remaining studies are thoroughly analysed to extract relevant information regarding the investigated topic, the methodologies employed, and their respective impacts on the research problem.

1.3 Methods

1.3.1 Eligibility Criteria

A set of constraints is defined to narrow the selection of papers related to the topic of interest. Table 1.1 presents the main characteristics that a paper must meet to be identified in the database search and subsequently evaluated during the screening process.

Characteristic	Inclusion	Exclusion
Problem	Predictions in image-sequence-based problems	-
Intervention	Deep reinforcement learning using images as input	-
Type of studies	Conferences, journals, articles	Books, magazines
Language	English	Other languages
Year	2010-2025	Before 2010

Table 1.1: Eligibility criteria for identifying studies.

1.3.2 Information Sources

The research has been conducted using arxiv, IEEE Xplore and Scopus, starting from the following query:

- ("deep reinforcement learning" OR "deep Q-network") AND ("image sequence" OR "video games") AND ("policy" OR "action" OR "reward") AND ("image pixels" OR "input image")

1.3.3 Selection Process

1. Identification

- Records identified from databases : 108

2. Screening

- Records excluded after reading title/abstract : 89
- Records excluded after reading the content : 9

3. Included

- Records included for review : 10

1.4 Findings

The review produced a variety of studies addressing the application of RL and DRL to image-sequence-based problems, such as video games, providing a comprehensive overview of the current state of the art. The selected studies, summarized in Table 1.2, range from early applications of RL to specific environments to the development of novel methods aimed at improving computational efficiency, stability, and performance.

The two studies conducted by Mnih et al. [4][5] laid the foundation for Deep Q-Learning (DQL), an approach widely adopted across numerous applications [6][7][8][9]. In subsequent studies, Samuel Alvernaz et al. [10] and Daiki Kimura [11] investigated the use of autoencoders to address more complex tasks and to adapt state representations for compatibility with other machine learning techniques. The integration of object recognition and image saliency techniques was examined by Yuezhong Li et al. [12] and Victor Augusto Kich et al. [13], who applied these methods in DQL frameworks to enhance the quality of the observation space.

Beyond the studies included in the table, it is worth mentioning the work by OpenAI et al. [14], in which they achieved superhuman performance in the complex zero-sum, team-based game Dota 2 by utilizing direct game information rather than raw visual input. Another interesting study was conducted by Elizabeth Gilmour et al. [15], in which part of the input images was occluded to encourage the agent to focus on the most informative regions of the screen. In the work of Adam Tupper et al. [16], different types of auto-encoders are compared for the production of compact state representations, emphasizing the role of representation size in the quality of the final performance. Finally, Rajesh Kumar Ojha et al. [17] addressed the challenge of General Video Game Playing as a step toward General Artificial Intelligence, evaluating multiple RL agents across multiple video games.

Authors	Year	Problem	Intervention	Results
Volodymyr Mnih et al. [4]	2013	Atari games	Invention of DQN	Good results using raw pixels as input
Volodymyr Mnih et al. [5]	2015	Atari games	Introduction of target network to DQL	More stable training of the DQN
Samuel Alvernaz et al. [10]	2017	DOOM video game	Auto-encoder and CMA-ES evolutionary algorithm	Worse than DQN but successfully able to use evolutionary methods
Daiki Kimura [11]	2018	Video games and real time images	Deep auto-encoder and QN	Improvement to DQN in complex tasks
Yuezhong Li et al. [12]	2018	Atari games	Object recognition applied to DRL	Improvement in respect classical models
Adil Khan et al. [6]	2018	DOOM video game	DQN and DQRN from DeepMind	Struggle with partially observable states
Ithan Moreira et al. [7]	2020	Robotic task	Interactive DRL	Much faster training but can lead to biases
Jing Xin et al. [8]	2021	Robot path planning	DQN with images	Good results using DeepMind Lab platform
Victor Augusto Kich et al. [13]	2022	Neon Drive video game	Low dimensional observation filter for DQN	Complete the task, but advantages are unclear
Advait Kulkarni et al. [9]	2023	Chrome Dino game	DQN from DeepMind and PPO	Problems related to real time constraints

Table 1.2: Included studies after the screening.

1.5 Discussion

From the conducted research, an increasing number of studies on this topic can be observed in recent years, along with a clear trend toward optimizing the computational time and memory requirements of large RL models. Many works demonstrate the feasibility of current DRL approaches to operate directly on image-based input data, highlighting the ability of agents to solve even complex tasks by exploiting the graphical elements present in visual observations.

The main advantages of using raw pixels as input include the independence of models from hand-crafted features in the observation space, the integration of RL systems into existing applications that cannot provide structured information to the agent, and the intrinsic similarity to human perception, which facilitates the interpretation of the agent’s decision-making process.

Most studies rely on variations of Deep Q-Network (DQN), a well-established method that combines Deep Neural Network (DNN) with Q-Learning (QL) to approximate an effective optimization policy. To a lesser extent, other RL approaches, such as actor-critic methods and evolutionary algorithms, are employed to address specific tasks or to accelerate learning. More recently, several works have incorporated auto-encoder-based techniques to reduce the dimensionality of image representations, enabling compact latent encodings to serve as inputs to DRL algorithms.

In scenarios with strict real-time constraints, various optimization strategies are adopted to reduce computational costs. The most impactful techniques include frame skipping in image streams, simplifying the input state through image preprocessing, and generating compact state representations directly from raw pixel data.

When dealing with high-dimensional observation spaces such as images, many studies also integrate attention mechanisms or specialized encoders to filter irrelevant information and focus on the most salient features.

Nevertheless, numerous works either avoid large state spaces altogether or report poor performance when applying traditional RL methods to such environments. To address these limitations, information about the environment is often manually provided to the agents, or exploration is artificially guided rather than left fully autonomous. While these strategies can improve performance, they introduce bias and may limit the discovery of innovative solutions.

During the development of this work, the interest in discovering new learning techniques shifted toward a more comparative approach, in which more basic methods are applied to increasingly complex problems to better understand their potential and to identify general characteristics that can be transferred to evolutions of these methods, particularly those designed to operate directly on images.

Chapter 2

Preliminaries

To ensure clarity and a proper understanding of the concepts discussed throughout this thesis, this chapter provides an introductory overview of the common terms and terminology used in the present work, as well as background knowledge on Reinforcement Learning (RL), with a particular focus on Q-Learning (QL) and Deep Q-Learning (DQL), the two subcategories introduced in Introduction. This chapter also outlines the main characteristics and challenges of these techniques, which will be revisited in later chapters.

Furthermore, a complete description of the virtual tools used during the implementation is provided, together with the motivation for their selection, in order to facilitate the reproducibility of the experiments in future work.

2.1 Reinforcement Learning

The core idea of RL [18] is to train an agent to perform a task, typically formulated as an optimization problem, by allowing it to interact with an environment and receive rewards based on the actions it takes (Fig. 2.1). Through many iterations, the agent constructs a value function, which estimates the expected return for each state-action pair. This function, combined with an internal policy, guides the agent toward making better decisions within the environment.

While the specific methodologies of different RL approaches may vary, the underlying principles and terminology used to describe the various elements remain the same. These terms, introduced in the following glossary, are intended to familiarize the reader and facilitate the explanation of the concepts presented in this thesis:

- **Environment:** A dynamic system that evolves over time and responds to actions produced by autonomous agents.

- **Agent:** A virtual entity responsible for interacting with the environment by executing actions and observing the resulting states.
- **State:** The combination of all observable parameters perceived by the agent after each interaction with the environment.
- **Action:** The mechanism through which the agent interacts with the environment, typically represented as a discrete integer.
- **Reward:** A numerical signal received after performing a specific action in a given state; positive for good actions, negative for bad ones, or zero otherwise.
- **Policy:** A strategy that defines which action to take given the current state.
- **Value Function:** An estimate of how beneficial a particular state is in terms of the expected future reward.

The fundamental idea of a RL agent is that, at the beginning, it is typically unaware of the relationships between states, actions, and rewards, and is not directly instructed on how to solve the problem, as in a traditional algorithm, nor on how to correct its errors, as in many supervised learning approaches. Instead, it relies on exploration and evaluation through a process of trial and error to iteratively construct its policy.

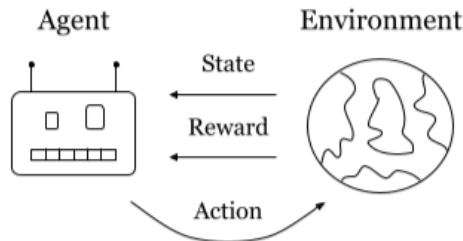


Figure 2.1: Interaction of an agent with the environment.

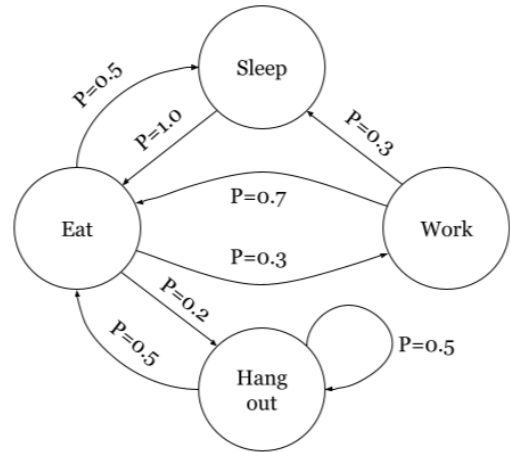


Figure 2.2: Example of a simple Markov Decision Process.

What provides RL with its theoretical foundations are the concepts of the Markov Property and Markov Decision Process. The Markov property states that

the future evolution of a system depends only on the current state and the action applied to it, and not on the past history. This means that a system satisfying this property is fully characterized by its current state, even though its evolution may still be stochastic. For environments that satisfy the Markov property, it is possible to formulate a RL problem as a Markov Decision Process (Fig. 2.2), in which each state-action pair (s, a) has predetermined probabilities of leading to next states s' with rewards r .

In complex environments, identifying the remote cause of an error can be challenging, as the actions that led to failure may have occurred many steps earlier. A similar difficulty is encountered by autonomous agents in reinforcement learning when rewards are too sparse. Assigning rewards only to positive outcomes or sporadic milestones can result in extended sequences of states with no feedback, commonly referred as sparse reward. In such environments, the agent receives limited information about the consequences of its actions and may struggle to properly evaluate intermediate state-action pairs.

Ideally, a reward signal should be assigned after every action, providing positive or negative feedback to the agent.

2.1.1 Q-Learning

QL [19] is an off-policy RL method that allows a separation between the acting policy and the learning policy. This separation enables the evaluation of the current state-action pair based on the result of the action taken in the subsequent state. To implement this mechanism, QL uses a value function $Q(s, a)$, which depends on the reward received at the current step and the maximum expected value of the next state. This function is updated according to the Bellman equation, as shown in Equation (2.1).

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)) \quad (2.1)$$

In this equation, r refers to the reward received after taking action a in state s . The parameter $\gamma : [0, 1]$ is the discount factor, which determines the importance of future rewards, and $\alpha : [0, 1]$ is the learning rate, which controls how much the value is updated at each step. Assuming $\alpha = 1$, and since $Q(s', a')$ is itself a Q-value, the same update formula is recursively applied to all future state-action pairs as follow.

$$Q(s, a) = r + \gamma \cdot \max_{a'} (r + \gamma \cdot \max_{a''} (\dots \max_{a^{n-1}} (r + \gamma \cdot \max_{a^n} Q(s^n, a^n)))) \quad (2.2)$$

The term $\max_{a^i} Q(s^i, a^i)$ in the Equation (2.2) is precisely what reinforces the evaluation of the Q-value, as it indicates the maximum value achievable in the

next state after taking an action in the current state. This intrinsically recursive structure allows each Q-value to be expressed as a weighted sum of all future Q-values, thereby producing the characteristic far-sighted behaviour.

Ultimately, an optimal policy seeks to maximize the total expected reward by selecting the action with the highest corresponding Q-value, which already accounts for the expected quality of future states.

All Q-values produced during training are stored in a data structure called the Q-table, which is updated each time a state-action pair is visited and a new Q-value is computed.

The Q-table essentially maps states to their associated Q-values and is often referred to as the “brain” of the agent. During the exploration of the environment, it accumulates all observed states and all possible actions that can be taken in those states. For each state-action pair, the table stores a value representing the expected benefit of taking that action in the given state. From a programming perspective, each value can be accessed as:

```
q_table[state][action] = q_value
```

The size of the Q-table is proportional to both the state space and the action space, and it can become extremely large in complex environments.

During training, the agent aims to explore every possible state-action pair and assign it a value. These values are reinforced each time the same state-action pair is revisited and are subsequently used to identify the optimal sequence of actions that leads to the highest total expected reward.

Because QL heavily relies on a tabular representation of discrete state-action pairs, in which states are treated as either identical or distinct, it lacks the ability to perceive similarity between states. This prevents generalization and limits its applicability to large or continuous state spaces; as a result, the method is typically used in small state-action spaces, where the number of possible states is limited and exploration is relatively fast.

Despite these limitations, its straightforward interpretability makes the agent’s decision-making and the evaluation of state-action pairs easy and transparent for a human developer to understand.

2.1.2 Deep Q-Learning

DQL, first introduced by Mnih et al. [4] and later refined by the same authors [5], is a RL technique that combines traditional QL with deep learning, enabling the use of more complex state representations and improved generalization. In their original work, the authors trained an agent to play various Atari games using raw pixel frames as input states.

Instead of a classical Q-table, DQL employs a neural network to predict the Q-values for all possible state-action pairs. This approach can handle much larger observation spaces and enables the agent to make reliable predictions even for states not encountered during training.

The two essential components that make DQL work are a neural network, known as the Deep Q-Network (DQN), which approximates the value function, and a replay buffer, which stores the agent's past experiences (state, action, reward, next state) in the form of transitions.

The observation space largely dictates how the network should be structured to correctly predict the Q-values for the possible actions. For observations composed of a series of simple variables, the network can be relatively small, reducing training time and preventing over-fitting. In contrast, for much larger observation spaces, such as raw-pixel images, the network must be larger and may incorporate different types of layers and operations to generate a compressed representation that can be used to extract meaningful Q-values.

The key feature that distinguishes the DQN from a standard supervised learning model is the computation of the loss for the training samples. In supervised learning, the loss is typically evaluated by comparing the network's predictions with a given ground-truth. In DQL, however, the ground-truth is not directly available and must be computed using the Bellman equation (2.1) applied to each transition. Whereas in tabular QL the maximum Q-value of the next state is stored in the Q-table, in DQL this value is predicted by the network itself by providing the next state as input and extracting the corresponding output. The target value is then determined using the expression:

$$\text{target} = \text{reward} + \text{discount_factor} * \text{best_predicted_q_value}$$

In more recent implementations [5][20], the target value is computed separately by another network called the target network. This network has the same structure as the policy network, and its weights are periodically updated by copying the parameters from the policy network after a fixed number of training steps. In this way, the target network provides the policy network with a more stable target, improving the stability of training.

Although the ultimate goal of the agent is to select the action that maximizes the expected reward for a given state, DQN does not perform a classification task. Instead, it performs a regression task in which the output is a vector of continuous values representing the Q-values for all possible actions, and the optimal action is determined afterward as the one with the highest Q-value.

The replay buffer, on the other hand, is used to facilitate the training of the DQN. It stores transitions and accumulates them to generate the dataset needed to train the network. Each time a new transition is observed by the agent, it is

added to the replay buffer. After a given number of steps, a random batch of transitions is sampled from this buffer and used to train the neural network. This mechanism helps break the temporal correlations between consecutive experiences and reduces the number of required interactions with the environment.

2.2 State-Space Trade-Off

As mentioned, a RL agent learns a policy within a specific environment based on the feedback resulting from its actions. This feedback is partly composed of artificial reward and partly of information about the most recent perceived state. This state consists of the subset of variables that the agent is able to acquire from the environment through its sensors or by direct injection from other sources.

To control computational complexity in experimental settings, the agent's perception is typically reduced, and therefore its representation provides only a partial approximation of the complete environment. Naturally, agents with different perceptual capabilities develop different policies. For instance, an agent that has access only to its absolute position can navigate a static environment with ease but cannot generalize the concept of a moving obstacle that appears in different positions over time. On the other hand, an agent that has access to all relevant environmental variables is able to perform a complete assessment at each decision step. However, the time required to explore and learn from all possible states may become infeasible in realistic scenarios.

This trade-off can lead to the partial observability of the problem and explain why an optimal solution for a given state representation may still be suboptimal with respect to the theoretical optimum achievable with full observability.

In practice, simple agents can acquire only a limited amount of information about the environment, while more advanced agents are able to perceive it almost completely. Generally, increasing the amount of information available to an agent can help it learn more effective behaviours and improve adaptability to changing environments. On the other hand, providing the agent with a large amount of data, in addition to increasing the time required for exploration and training, can inhibit the agent's ability to identify the most relevant variables among the many available.

Another important consideration is that the number of possible values an environmental variable can assume directly affects the complexity of the state space, with more values making the problem harder to solve.

In QL for example, the agent needs to explore every possible state to be fully certain of its solution. For small state spaces, this process is trivial, and the solution can be achieved in a short amount of time. At the other end of the

spectrum, if the state variables are continuous or unbounded, a definitive solution cannot be achieved in a finite amount of time. This issue arises directly from the number of values that a single state variable can assume and can only be addressed by either manually adjusting the agent's perceived state to the nature of the problem or by using stochastic approaches to build a policy that identifies correlations between similar samples and approximates the variability of the states.

2.3 Exploration vs. Exploitation

A distinctive characteristic of any RL method is the presence of the exploration-exploitation dilemma. To learn effectively, an agent must explore a wide range of states to evaluate different state-action combinations. At the same time, however, it must exploit the best-known policy to reach further states and optimize the solution. This dilemma becomes especially apparent in problems involving a large number of states, intermediate terminal states, or states that are only accessible through specific progressions, and it is one of the main challenges in designing autonomous agents.

A common approach to address this issue, also adopted in this implementation, is the use of the ϵ -greedy action-selection policy, which controls the probability of taking a random action through the parameter ϵ . At each step, by randomly sampling a decimal number from a uniform distribution, if this number is less than ϵ , the agent selects a random action, otherwise it follows the best policy learned so far. In practice, if ϵ is set to 1, the agent behaves in a completely random manner, while if ϵ is set to 0, the agent relies entirely on the learned policy. Typically, ϵ is set to zero once training is completed and the agent is ready to be freely deployed in the environment.

The simplest implementation of this method involves setting ϵ as a constant, resulting in a fixed probability of choosing a random action. This approach is useful for simple problems in which the number of possible states is small and exploration is not particularly critical for the agent.

A slightly more advanced technique gradually decreases the value of ϵ as the agent learns an optimal policy. This allows the agent to explore more extensively at the beginning of training and become increasingly confident of its actions as learning progresses. This version works well in stable and continuing environments, where the agent can explore indefinitely at each iteration without intermediate terminal states.

Although these strategies perform well in simpler environments, they may be insufficient when the state space is large or complex.

In continuous learning scenarios, where the agent keeps exploring new states indefinitely, completely suppressing exploration over time is generally not advisable, as important states may still be discovered in the future. Similarly, insufficient exploration of early states can compromise the agent’s ability to learn subsequent states effectively. Therefore, adopting the previously mentioned strategies in such environments may result in suboptimal learning.

In such scenarios, the agent must adapt its learning strategy based on its training progress and the performance reflected by the current evaluation metrics.

Another challenge that may arise in certain RL approaches, such as DQL, involves the agent’s ability to map different states, which can be overwhelmed by the large number of training samples, potentially leading to catastrophic forgetting, in which previously learned values for some state-action pairs are overwritten by new, more frequent patterns, causing degradation of the policy.

To address these challenges, a custom adaptive ϵ -greedy policy is presented in Section 3.2, which maintains a balance between exploration and exploitation.

2.4 Tools and Libraries

The structure of the RL environment is based on the framework proposed by OpenAI in their Gym library. Initially, Gym was used to gain familiarity with RL programming, but it was later replaced with a more customized approach.

PyTorch is a popular Python library that facilitates deep learning with high flexibility and customization. It is centred around the concept of tensors and operations on them. PyTorch allows constructing neural networks from scratch, including all components such as loss functions, custom layers, and datasets. At the same time, it provides a wide range of commonly used modules and functions that speed up the implementation process. PyTorch has been instrumental for the implementation of the custom DQN and its various components.

Standard Python libraries such as Matplotlib and Pickle were also used for data organization, visualization, and result storage.

2.4.1 G2D Library

G2D [21] is a graphics library for Python. It was used to implement the rendering component of the game and to obtain visual snapshots of the environment during the training process. The library allows drawing basic shapes and lines on a canvas, writing text, adding interactive buttons, and importing images and sounds in various formats. Its functioning is based on a main loop that sequentially draws elements on the screen, overlays them on top of each other, and then clears and refreshes the canvas at each iteration.

Chapter 3

Implementation

3.1 Environment

The player character moves within a two-dimensional plane while being subject to simulated gravity and interacting with the environment through a collision system. The game consists of a single level in which the positions of all objects are fixed and remain constant over time. Certain objects may be removed from the environment following collisions with the player character. The level layout and game parameters can be configured through an external configuration file to adjust the complexity of the environment and the duration of gameplay.

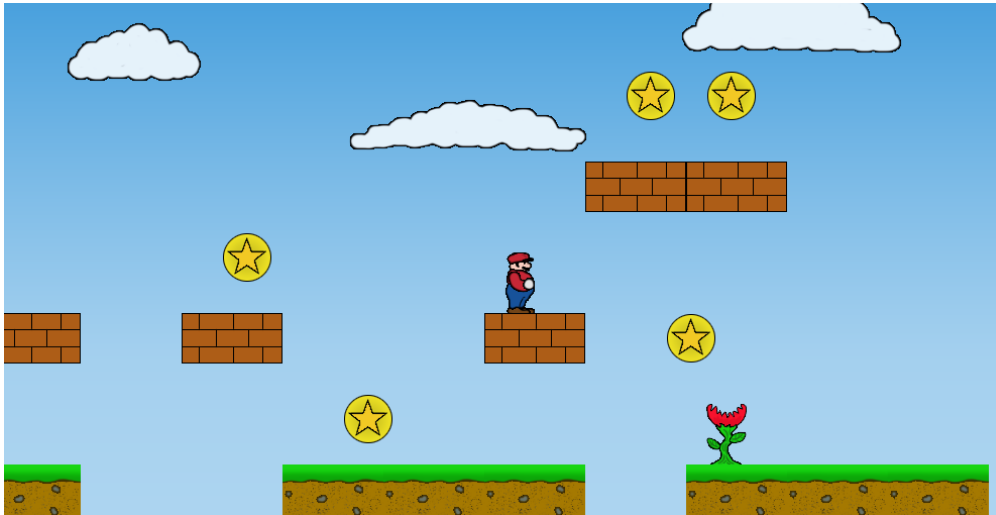


Figure 3.1: Screenshot of the environment taken during gameplay.

Distances within the environment are measured in pixels, which are used both in absolute coordinates (with respect to the top-left corner of the map) and in rel-

ative coordinates (with respect to the top-left corner of the display window). Due to implementation constraints, all implemented objects have rectangular hitboxes, which may result in collisions even when the corresponding visual models do not appear to intersect.

To ensure seamless integration between the reinforcement learning framework and the game engine, the environment exposes a set of functions that provide access to relevant metrics and parameters, including object positions, score values, game state information, and gameplay time.

The environment can operate in two different modes. The first mode renders each step of the game on the screen, allowing a human player to play or visually evaluate the agent’s performance. The second mode suppresses screen output and runs computations only, which accelerates training.

The game environment not only makes experimentation more engaging but also provides an abstract representation of a real-world environment in which an intelligent agent can interact with objects, avoid obstacles, operate under physical constraints, and pursue specific goals.

Testing in such simulated environments offers a practical and safe alternative to deployment in real-world scenarios, as it reduces practical risks and significantly lowers costs compared to high-fidelity simulations or real-world experimentation, which are often impractical in academic contexts.

3.1.1 Actors

The game is developed in Python using the object-oriented paradigm, in which each class defines the attributes and behaviours of the game elements. In this implementation, an actor is an independent entity that can interact with other actors within the system. The abstract class **Actor** (see Fig. 3.2) is extended by multiple subclasses, each of which implements the attributes and methods characteristic of a specific element of the environment. Each object is then managed by a main coordinating entity.

The **Arena** class serves as the main coordinator of the game, managing all actors and their interactions. It maintains a list of all actors created before and during the game and provides methods to manage actors and collisions between them, register user inputs, update actor positions, and render images on the screen, as well as multiple auxiliary methods required for the game logic and for returning game parameters. The **Arena** also exposes API methods for interaction with various Reinforcement Learning (RL) applications, such as sharing environment states and receiving actions as inputs.

The **Player** class manages the player character and represents an active element of the game, as it can move the character within the environment and handle

Actor
+ relative_move() : None + absolute_move(x : float) : None + draw() : None + position() : tuple + size() : tuple

Figure 3.2: Representation of the abstract class *Actor*.

collisions with most other objects. It also allows reading the player’s keyboard inputs to control the character’s movements or, alternatively, to receive actions as values between 0 and 3 via an API method.

Other classes are implemented as subclasses of **Actor** to represent the remaining elements of the game. Each class can be instantiated multiple times and added to the **Arena** to customize the environment.

3.1.2 Game Mechanics

During the game, the player can perform three primary actions to navigate the environment. These actions include moving forward, moving backward, and jumping, in addition to the simple action of standing still without providing any input. While in the air, the player can move freely in both horizontal directions to adjust the trajectory of the jump.

As the player progresses through the level, they encounter various obstacles that make reaching the end more challenging. These obstacles include elevated platforms, gaps in the terrain, and dangerous plants. To succeed, the player must avoid touching the plants and falling into the gaps; otherwise, the player character dies and must restart from the beginning.

Throughout the level, coins are placed in the environment to reward the player. Collecting a coin increases the player’s score, which is displayed at the top of the screen.

The ultimate goal of the player is to reach the end of the level, marked by a flag, while collecting as many coins as possible.

3.1.3 Graphics

The graphical elements of the game are rendered on screen using a simple frame-based rendering pipeline. At each frame, the previous canvas content is cleared

and replaced with a newly composed image representing the updated game state. Before rendering, the properties of all game objects are updated to reflect the current frame logic.

The player character features two distinct animations: one for running and one for jumping. During horizontal movement, the player remains horizontally centered on the screen. Instead of moving the player model along the x-axis, all other environmental elements are translated in the opposite direction, creating the illusion of forward motion while the player appears stationary.

During a jump, the environment remains anchored, and the player model moves vertically along the y-axis according to the jump physics.

At the top of the screen, a user interface displays key gameplay metrics, including the current score, the maximum distance reached during the run, and the time that has passed since the beginning of the level.

When the run ends, either due to the player's death or upon reaching the end of the level, the final metrics are shown on screen, and the player is then given the option to restart the game.

3.2 Exploration Policy

In large state-action space problems, such as the one addressed in this thesis, the exploration-exploitation dilemma discussed in Section 2.3 has a significant impact on the agent's ability to learn an optimal policy. In the studied environment, the agent must progress through the current level to discover new states that are not known at the beginning. This requirement limits the effectiveness of the traditional ϵ -greedy strategy, in which the probability of taking random actions gradually decreases over the course of training.

To address this issue, a newly-proposed algorithm has been implemented (see Algorithm 1), in which the probability of random actions is adjusted dynamically at both the episode and generation levels. After each generation, the agent memorizes the maximum distance reached during the previous generation's test run and sets this value as an upper bound. This upper bound represents the point that can be reached by relying solely on the exploitation of the current policy and is considered a confident estimate of progress.

In the following generation, steps taken within this upper bound are treated as trivial, and a lower ϵ value is used to reduce, but not completely suppress, exploration of the environment. If the agent manages to exceed the upper bound, the value of ϵ is increased to encourage further exploration.

To prevent situations where the agent becomes stuck near the boundary of the upper limit, a linear transition mechanism has been introduced to smoothly interpolate between the two ϵ values (Fig. 3.3). In this way, if the upper bound

is situated near a terminal state with no possibility of recovery, the agent is still able to explore alternative actions before reaching that state.

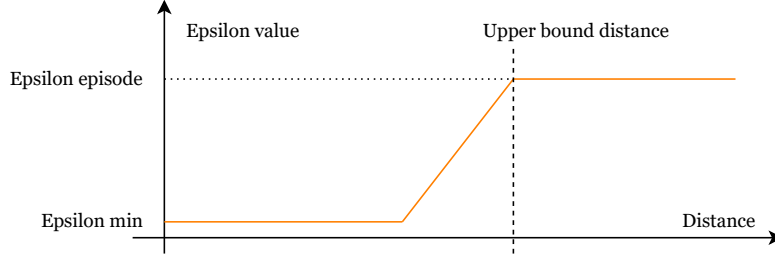


Figure 3.3: Adaptive ϵ -greedy behaviour based on distance.

Finally, to further accelerate the learning process, the value of ϵ above the upper bound is decreased by a small amount after each episode, with the decrement proportional to the number of episodes in a single generation. At the end of each generation, ϵ is reset to its maximum value. This strategy allows the agent to gradually become more confident in its learned policy within a generation while maintaining sufficient exploration.

Algorithm 1 Epsilon Value Computation

```

for each episode do
  for each step do
    Set the interpolation starting_point: upper_bound - interval
    if current_distance  $\leq$  starting_point then
      Set epsilon to epsilon_minimum
    else if current_distance  $\geq$  upper_bound then
      Set epsilon to epsilon_episode
    else
      Interpolate between epsilon_minimum and epsilon_episode
    end if
  end for
  Decrement epsilon_episode by epsilon_decay
end for

```

During training, it is important to maintain the value of ϵ always greater than zero. Even when the agent is confident in its learned policy, small amounts of exploration may still lead to alternative, more optimized paths. Furthermore, in problems with large state spaces, it is recommended to keep the maximum value of ϵ below one to prevent fully random behaviour, which can significantly slow down the training process.

In particular, in Deep Q-Learning (DQL) approaches, since the initialization of the network weights is already a stochastic process, the predictions are inherently affected by randomness. Therefore, the maximum value of ϵ is further reduced (see Tab. 4.2).

3.3 Reward Function

Reward engineering represents one of the most critical steps in the development of an effective RL model. The choice of which actions should be positively or negatively rewarded, as well as the magnitude of these rewards, can significantly influence the success or failure of the learning process.

In the considered environment, most of the game objectives already provide direct feedback to the player through a scoring system. Advancing through the level, collecting coins, and reaching the end of the stage all contribute positively to the overall score. For this reason, these elements are naturally incorporated into the reward function, as they align well with the intended objective of the task. As discussed in Section 2.1, sparse reward is a well-known issue in reinforcement learning that can hinder consistent learning. To mitigate this issue, a small positive reward is assigned at every forward step rather than only at predefined milestones. This provides the agent with a consistent learning signal for positive behaviours.

While positive reinforcement promotes desirable behaviour, it is equally important to discourage suboptimal or harmful actions. Behaviours such as falling into a hole or remaining idle can work against the success of task completion and must therefore be penalized through negative rewards.

The main challenge in this context arises when a negative reward is assigned exclusively at the end of an episode since the agent may experience difficulty in perceiving the causes of failure. In fact, the agent must rely on temporal-difference updates derived from the Bellman equation (Equation (2.1)) to propagate negative signals backward through preceding states. The discount factor γ determines how strongly future states influence the current value estimate. However, even with high values of γ , the impact of distant negative signals vanishes after a certain number of steps, making it difficult for early states to properly reflect their contribution to failure.

This issue does not admit a universal solution and must be addressed according

to the specific characteristics of the environment. In the present implementation, a negative reward is assigned as soon as the agent initiates an irreversible failure, for example when the character begins falling into a hole. By anticipating the terminal event, the delay in the negative learning signal is reduced. Although this approach does not capture every possible failure configuration, such as falling from high platforms, it significantly improves the correlation between failure and cause in most cases.

The reward function presented in Algorithm 2 summarizes all configurations in which the agent receives feedback. Since the environment remains unchanged across experiments, the structure of the reward function is fixed, with only minor adjustments to reward magnitudes.

Algorithm 2 Reward Function

```

if Game is running then
  if No progress since  $K$  episodes then
    Reset game
    return  $-R_{stalling}$ 
  end if
  if Stationary on ground then
    return  $-R_{stalling}$ 
  end if
  if Falling into a hole then
    Reset game
    return  $-R_{death}$ 
  end if
  return  $(progress \cdot R_{progress} + score \cdot R_{coin})$ 
else
  if Level complete then
    return  $R_{victory}$ 
  else
    return  $-R_{death}$ 
  end if
end if

```

Although the game already provides values for some evaluation metrics, converting them into a suitable reward signal is more complex than simply using the numerical values output by the environment. This is primarily due to a scaling issue between multiple objectives. The reward values must be carefully tuned

to prevent the agent from over-focusing on a single objective, ensuring that each objective is considered appropriately. Additionally, as mentioned, different approaches require different reward scales to better align with the method employed (see Table 3.1).

In Q-Learning (QL), large reward values have been used to emphasize the primary objectives and to increase the magnitude of the signal propagated backward to previous states via the Bellman equation. In contrast, in DQL, the reward is directly used to generate target values for computing the training loss over batches of transitions. Therefore, the reward values returned by the environment have been scaled to the range $[-1,1]$ as a form of normalization, reducing gradient variance between samples and resulting in more stable training.

Method	Death	Victory	Progress	Coin	Stalling
Baseline Q-Learning	-1000	1000	1	10	-1
Perceptive Q-Learning	-1	100	0.0001	100	-0.01
Perceptive Deep Q-Learning	-1	1	0.1	1	-0.01
Visual Deep Q-Learning	-1	1	0.01	1	-0.01

Table 3.1: Reward values for each experiment.

3.4 State-Action Memory

For a RL agent, pairing states, actions, and rewards is an essential aspect of the learning process, as it allows the agent to establish a causal relationship between its observations and its behaviour. To support this, each agent implements a data structure representing the value function, which associates each state-action pair with a quantitative value used to evaluate the policy.

In traditional Q-learning applications, this structure is most commonly implemented as a multidimensional table that contains all possible state-action pairs, with initial Q-values set to zero. The entries of the table are then updated whenever the corresponding state-action pair is visited.

Initializing all possible states, however, requires perfect prior knowledge of the environment, which represents a strict constraint in many continuous or large-scale learning applications. Furthermore, in environments with an unbounded or very large number of states, the memory demand to store all the possible states significantly increases, making the use of a conventional Q-table infeasible.

For example, in an environment of size 5000×700 pixels where the state is defined only by the absolute position of the player in Cartesian coordinates, and four actions are possible in each state, the Q-table would require approximately

14 million entries ($4 \cdot 5000 \cdot 700$), which is already a substantial memory demand. Incorporating additional variables into the state representation quickly increases the table size, resulting in memory usage and training times that may exceed the capabilities of modern hardware.

Moreover, in most problems with large state-action spaces, the agent explores only a small fraction of all possible states due to the exploration-exploitation trade-off (see Sec. 2.3). As a result, the full Q-table often overestimates the portion of the state space that is actually relevant.

To address these issues, this work proposes a more efficient strategy in which the Q-table is constructed dynamically as the agent explores the environment. With this method, only visited states are included in the table, allowing for a richer state representation without exponentially increasing memory requirements.

```

1
2 while True:
3
4     if state not in q_table():
5         q_table()[state] = [0]*env.act_space_size
6
7     q_values = q_table()[state]
8
9     ...
10
11    action = select_action(...)
12
13    next_state, reward, terminated, infos = env.step(action)
14
15    q_value = q_values[action]
16
17    if next_state not in q_table():
18        q_table()[next_state] = [0]*env.act_space_size
19
20    best_q = max(q_table()[next_state])
21
22    q_delta = lr * (reward + discount * best_q - q_value)
23    q_table()[state][action] += q_delta
24
25    state = next_state
26
27    ...
28

```

Figure 3.4: Example showing the use of the compact Q-table in the training cycle.

The compact Q-table is implemented as an empty dictionary. Each time a new state is encountered, a new key is created in the dictionary and assigned a list of zeros corresponding to the number of possible actions. If the state is already present in the dictionary, the corresponding Q-values are retrieved from that key.

After an action is selected and the resulting reward is observed, the Q-value corresponding to the selected state-action pair is updated in the dictionary, as illustrated in Figure 3.4.

This dynamic implementation of the Q-table significantly reduces memory requirements while enabling faster lookup and update of specific states.

As explained in Section 2.1.2, DQL eliminates the need for a Q-table by employing a Deep Q-Network (DQN). However, to train the network effectively, the agent must still record its interactions with the environment. This requirement is addressed with the Experience Replay mechanism illustrated in the next section.

3.4.1 Experience Replay

In the case of Experience Replay (or replay buffer), as used in DQL approaches, the memorization of transitions is only relevant during the training process, in which the agent learns patterns in the data to predict their Q-values. For this reason, the memory must be managed at runtime, with the ability to insert new states and remove older ones. To achieve this, the replay buffer is implemented as a Python `deque`, a FIFO (first-in, first-out) data structure in which older samples are automatically discarded when capacity is reached to make room for new ones. The replay buffer stores transitions recorded during the agent's exploration and can return batches of 64 elements for use in training the DQN.

To train a DQN, the dataset must be sufficiently large to prevent over-fitting and the forgetting of previously learned correct behaviours. For this reason, the buffer size is set to 10^6 samples.

A more advanced experience replay, proposed by Tom Schaul et al. [22], uses the magnitude of a transition's temporal difference (TD) error to determine the importance of samples in the buffer. Highly valuable samples have a higher probability of being selected in a batch when training the network, ensuring that important but under-represented transitions are learned effectively.

Similarly to this approach, a couple of sampling techniques are tested in this work, evaluating the impact of each one on the studied problem.

3.5 Baseline Q-Learning

The baseline experiment represents the starting point of this research. Its objective is to evaluate the feasibility of applying classical QL to the studied problem and

to gain familiarity with the fundamental concepts of RL. Assessing this method is also useful for identifying the limitations of Q-learning in terms of performance and scalability.

In this approach, the agent relies exclusively on basic information about its current position in the environment, without any additional contextual awareness. Its strategy consists of learning a sequence of state-action pairs that lead to the final objective, using the reward mechanism described in Section 2.1.1. Since the agent is unable to perceive physical elements or dynamic changes in the environment, it is particularly sensitive to situations in which the same state may produce different reward values at different times. For example, a coin’s position may correspond to different rewards depending on whether it has already been collected, introducing ambiguity into the learning process.

The implementation adopts the Q-table approach described in Section 3.4 to store the Q-values associated with each state-action pair. These values are updated during the agent’s training process, which is organized into episodes and generations to structure the learning procedure.

An episode corresponds to a complete run in the environment, characterized by the maximum distance reached and the total accumulated reward. A generation consists of a fixed number of episodes, at the end of which a test run is performed to evaluate the agent’s progress and to update the upper-bound used in the exploration policy described in Section 3.2. Additionally, after each generation, the Q-table is saved to file as a checkpoint. This is made feasible by the reduced representation of the Q-table adopted in this implementation.

During an episode, each step taken in a given state produces a reward and a subsequent state. The next state is then used to compute the maximum expected future Q-value, and the current Q-value is updated according to the Bellman Equation (2.1).

At the beginning of training, or prior to testing a model, the agent can be initialized by simply loading the saved Q-table representation from the file.

3.5.1 State Space

The state space adopted in the baseline approach is intentionally simple. Each state corresponds to the Cartesian coordinates of the agent’s position within the environment, sampled at pixel resolution. Consequently, every pixel in the game area represents a possible state in the state space.

The main advantage of using the agent’s position as the only state representation lies in its interpretability, as it makes the relationship between states and actions intuitive and easily understandable, even from a human perspective.

However, providing the agent with only positional information makes the prob-

lem highly partially observable, as important environmental characteristics are omitted from the state representation and, therefore, cannot be taken into account during decision making.

3.6 Perceptive Q-Learning

This approach aims to test the flexibility of a RL agent capable of perceiving objects in the environment rather than relying on fixed coordinates and unique states, allowing it to recognize recurrent patterns, generalize across states, and adapt to moving or temporary objects. In this setting, the state is no longer represented by a coordinate; instead, it is defined as a combination of values indicating the distance from specific features in the environment. To achieve this representation, the environment returns a hand-crafted state at each step, in which the distance in pixels from each predefined actor within a certain range is computed and normalized to a fixed interval of values.

This representation offers a noticeable improvement over the baseline, both in terms of the amount of information available to the agent and the memory required to store the states. This memory efficiency is primarily due to the scalability of the problem when the environment changes dimensions. By using this hand-crafted observation space, if the environment map expands while maintaining the same types of features, the state representation remains unaffected. In contrast, in the coordinate-based approach, the state space would increase proportionally with the environment map size. This behaviour reflects the state-space trade-off discussed in Sec. 2.2, where unbounded attributes can potentially cause the state space to grow toward infinity.

In this case, a trade-off for limiting the possible values m of the state attributes is to increase the number n of attributes that encode information about the environment. However, this solution introduces a memory challenge for storing the Q-values of the state-action space, since the effective state space has dimension m^n , meaning that the number of attributes has a greater impact on the memory requirements than the range of each attribute.

To partially address this issue, the compact Q-table structure described in Section 3.4 is employed, in which new states are registered at runtime, and the Q-values of existing states are updated as the agent continues to interact with the environment.

3.6.1 State Space

As mentioned in the previous section, the Perceptive Q-Learning Agent is able to perceive multiple environmental elements in the vicinity of the player model. These

elements (Fig. 3.5) include the platforms and terrain sections on which the player can stand, the coins yet to be collected, and the plants that must be avoided to prevent collisions. Additionally, a single binary variable is used to indicate whether the player is currently on the ground or in the air.

Attributes	In air	Previous ground	Next ground	Coin	Plant
Types	Binary	Float	Float	(Float, Float)	(Float, Float)

Attributes	Prev. top platform	Next top platform	Prev. bottom platform	Next bottom platform
Types	(Float, Float)	(Float, Float)	(Float, Float)	(Float, Float)

Figure 3.5: Environmental attributes composing the observable state.

The variables related to coins and plants are represented as tuples containing two values: the horizontal and vertical distances to the closest object of each type within the perception range.

Regarding platforms, four variables are provided simultaneously: two corresponding to platforms at the upper level and two to platforms at the middle level. For each height level, the two platforms considered are the one aligned with the player (previous platform) and the next platform within range (next platform). Each platform information is described by a tuple containing its horizontal and vertical distances relative to the player.

For terrain sections, only the horizontal distance information is provided. Similarly to the platform representation, two ground sections are considered in the state, representing the previous and the next section relative to the player's position.

The perception range is set to 500 pixels, which corresponds to approximately 70% of the display window width. This allows the agent to detect relevant objects in advance and adjust its behaviour accordingly.

If certain objects are not present within the perception range, their corresponding variables are assigned a default, extremely large value. A negative sign is used for variables representing previous platforms or terrain sections to preserve directional information.

To further reduce the size of the state space without excessively compromising the model's accuracy, each distance value is capped at half of the perception range and rounded to the nearest integer. This effectively halves the measurement interval and significantly reduces the number of possible state combinations.

3.7 Perceptive Deep Q-Learning

This experiment is based on the Perceptive Q-Learning environment described in Section 3.6, in which the agent was able to perceive distances from selected objects while adopting a different RL strategy based on DQL.

The observation variables and the reward function remain largely unchanged to facilitate a fair comparison between the two approaches, although minor adjustments were necessary to ensure stable and effective training of the model.

As discussed in Section 2.1.2, DQL extends traditional QL by replacing the Q-table with a DQN that approximates the Q-values of state-action pairs, while Experience Replay is used to process the agent’s experiences and stabilize learning.

The deep network employed in this approach is a Multi Layer Perceptron (MLP) architecture (see Table 3.2), composed of fully connected layers (Linear in the Table) trained via backpropagation and gradient descent.

Layer	Operation	Input	Output
1	Linear + Tanh	15	64
2	Linear + Tanh	64	32
3	Linear + Tanh	32	8
4	Linear	8	4

Table 3.2: Perceptive Deep Q-Network architecture.

A hyperbolic tangent (tanh) activation function is applied after each hidden layer to introduce non-linearity while allowing both positive and negative activations. Since rewards and Q-values may assume negative values, tanh is a more suitable choice than the commonly adopted ReLU, which sets negative outputs to zero (see Fig. 3.6).

To implement the target network described in Section 2.1.2, a second model identical to the policy network is initialized using the same random seed for weight initialization. As a result, both networks start with identical predictions and are then updated asynchronously to mitigate the instability caused by the moving target problem during training.

During training, the agent continuously interacts with the environment, similarly to the previous approach. At each time step, it observes the environment and retrieves the current state, selects an action according to its policy, and obtains a reward. Rather than using this experience immediately for learning, the interaction is stored as a transition (state, action, reward, next state) in the replay buffer.

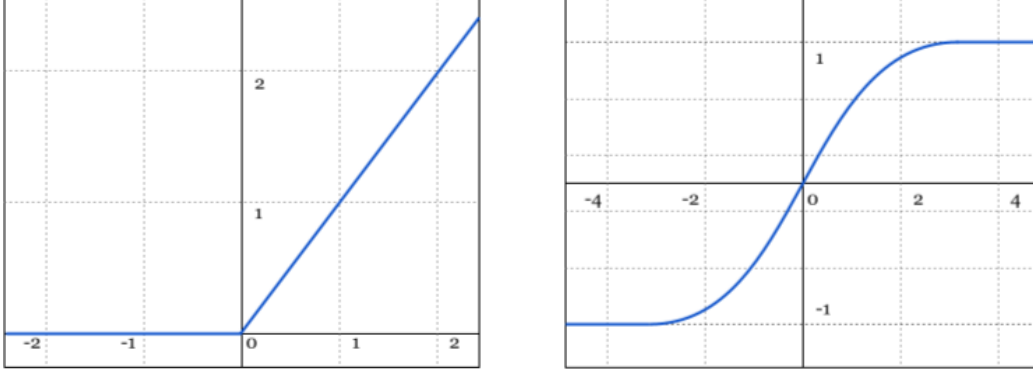


Figure 3.6: Comparison between ReLu (on left) and Tanh (on right) activation functions.

To ensure a sufficient number of recent and relevant experiences, the replay buffer is implemented as a fixed-size First-In, First-Out (FIFO) structure with a capacity of one million transitions.

The training of the network begins only once the replay buffer has accumulated a sufficient number of samples to enable the extraction of a batch of 64 transitions.

At each subsequent training step, a batch of samples is randomly drawn from the replay buffer, providing the states, actions, rewards, and next states. These states are then passed through the policy network in a forward pass to obtain the predicted Q-value for the action stored in each transition.

Afterward, the target Q-values are computed using the Double Deep Q-Network strategy [20]. The next states in the batch are first evaluated by the policy network to determine the target actions that maximize the predicted Q-values, and these target actions are then used to select the corresponding Q-values from the predictions of the target network. In this way, the policy network identifies the best next action while the target network provides a stable estimate of its value.

Using the computed target Q-values together with the rewards from the transitions, the final target Q-values are calculated as a slight variation of the Bellman equation (2.1), and the loss between the predicted Q-values and the targets is evaluated using the `SmoothL1Loss` function.

Finally, the gradient is computed for every network parameter, and the network weights are updated by performing a gradient descent step using the optimizer.

As discussed in Section 2.1.2, the target network must remain consistent with the policy network to provide reliable estimates of the target Q-values. Updating the target network at every step would eliminate the stability gained by mitigating the moving target effect. Therefore, the target network is updated only every 100 episodes by copying the weights and biases from the policy network.

```

1
2     policy_network.optimizer.zero_grad()
3
4     q_pred = policy_network(states)
5
6     q_pred = q_pred.gather(dim=1, index=actions)
7
8     with torch.no_grad():
9         target_act = torch.argmax(policy_network(next_states))
10        target_pred = target_network(next_states)
11        q_best = target_pred.gather(dim=1, index=target_act)
12
13    q_target = rewards + discount * q_best * (1 - terminals)
14
15    l = loss(q_pred, q_target)
16    l.backward()
17    policy_network.optimizer.step()
18

```

Figure 3.7: Simplified python implementation of the network training cycle.

3.7.1 State Space

The attributes that compose the state representation are identical to those used in Perceptive Q-Learning, as reported in Table 3.5, although their values differ in the new observation space.

By introducing a neural network to predict the Q-values, the input representing the observed states must be adapted to be compatible with the operations performed by the neurons. Values that were previously set to infinity to indicate objects outside the range of perception are now replaced with a fixed maximum value greater than the others. However, this is a compromise that incentivizes the network to treat these values similarly to regular inputs, which may sometimes lead to incorrect predictions.

As a final step to improve training performance and stability, the observation vector is normalized around the constant maximum value and its negative counterpart.

3.8 Visual Deep Q-Learning

In this approach, the agent relies exclusively on visual perception to learn the policy, without access to the internal states of the environment. The overall structure of the agent is very similar to that used in the Perceptive Deep Q-Learning

approach (Section 3.7). The main difference lies in the type of observed state and in the architectural components required to process it.

The central structural difference with respect to the previous approach is the network architecture, which must process image inputs rather than simple low-dimensional tensors. Recent advances in deep learning have demonstrated that Convolutional Neural Network (CNN)s significantly outperform MLPs on image-based tasks. For this reason, the implemented architecture follows that of a compact CNN composed of a single convolutional layer that takes as input a stack of three gray-scale images of size 84×84 and outputs a tensor of four values corresponding to the Q-values of the possible actions.

The network consists of five primary layers (see Table 3.3), which progressively compress the input into a compact representation suitable for Q-value prediction. The first layer is a custom module, **PatchLayer** (see Figure 3.8), designed to preserve spatial information. Standard CNNs are typically translation-invariant, focusing on detecting patterns regardless of their position. However, in Visual Deep Q-Learning, sensitivity to object position is crucial, as it directly influences the optimal action.

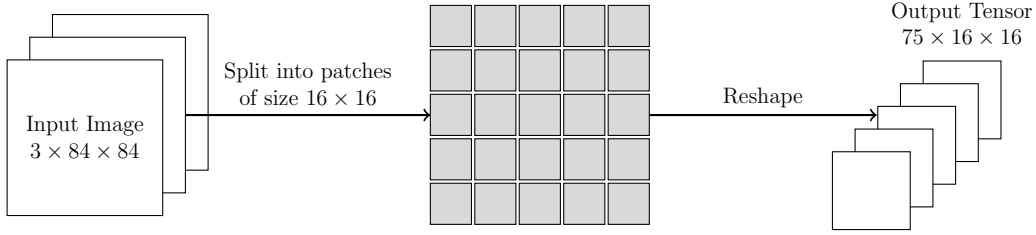


Figure 3.8: Visualization of PatchLayer operations.

The **PatchLayer** divides each input channel into a 5×5 grid of patches, each of size 16×16 , and stacks them along the channel dimension, producing a tensor of shape $75 \times 16 \times 16$. To stabilize training and accelerate convergence, this output is normalized using Batch Normalization, followed by a ReLU activation to introduce non-linearity. By doing so, the network explicitly encodes spatial information, allowing subsequent layers to focus on feature extraction within each patch.

The second layer is a convolutional layer that applies a kernel of size 5×5 with a stride of 3 and no padding. This operation reduces each patch to a 4×4 feature map for each filter, producing 32 feature channels that are then processed by Batch Normalization and a ReLU activation function, as in the previous layer. These feature maps are then flattened into a 512-dimensional vector, which is processed by two fully connected layers, with a ReLU applied after the first linear layer, to condense the information and output the four Q-values corresponding to

the possible actions.

Overall, this architecture ensures that the network maintains spatial awareness of the environment while progressively extracting high-level features, enabling the agent to take actions based on both object positions and visual patterns.

Layer	Operation	Input	Output	Details
1	PatchLayer + BN + ReLU	$3 \times 84 \times 84$	$75 \times 16 \times 16$	patch size: 16
2	Conv2d + BN + ReLU	$75 \times 16 \times 16$	$32 \times 4 \times 4$	kernel: 5 stride: 3
3	Flatten	$32 \times 4 \times 4$	512	-
4	Linear + ReLU	512	64	-
5	Linear	64	4	-

Table 3.3: Visual Deep Q-Network architecture.

The network is initialized with random weights, except for the final layer, whose weights and biases are initialized to zero. This ensures that the untrained model initially outputs a zero vector, analogous to the null initialization of a Q-table. During training, the network progressively adjusts its outputs, allowing it to favour better actions from the earliest learning iterations.

The training process follows that described in the Perceptive Deep Q-Learning approach (Section 3.7), using the same Double Deep Q-Network technique and the hyper-parameters reported in Table 4.2.

The same replay buffer implementation described in Section 3.4.1 is employed to store transition samples. In this case, however, the memory requirements are higher due to the image-based representation of the states.

3.8.1 State Space

Visual observations differentiate this approach from the other RL methods used in this study. In contrast to receiving a vector of internal game states, the agent now perceives the environment through raw images, similar to how a human player experiences the game.

At each frame, a portion of the screen is captured as a gray-scale image. This region, centred on the player character, is a square with a side length of 500 pixels, corresponding to approximately 35% of the area that a human player would typically see while playing. The extracted image is then resized to an 84×84 square, performing downsampling that reduces detail but improves storage efficiency and training speed. Areas extending beyond the display window are filled using zero

padding. Finally, the image is normalized to facilitate stable and faster convergence of the network.

To provide the agent with a better perception of motion and progression through the environment, three consecutive frames are stacked along the channel dimension, forming a three-channel representation of the current state (see Fig. 3.9).



Figure 3.9: Three consecutive frames stacked along the channel dimension generating the state.

Chapter 4

Experimental Results

This chapter presents the evaluation methodologies used during the training and testing of the models. In particular, it describes the setup of the experiments, depicts the metrics adopted to evaluate performance, and reports the most relevant results obtained for each approach.

Despite not always producing positive results, the experiments highlight the strengths and limitations of the explored approaches in large and partially observable state spaces and suggest potential directions for improvement.

Due to the stochastic nature of Reinforcement Learning (RL), derived from the exploration policy and the random parameter initialization of the network, the models obtained from training result in slightly different outcomes across multiple runs. To ensure clarity of presentation, the figures report the results of a single representative run, while the average performance across multiple runs is analysed and discussed separately.

All models were trained and tested on a local machine equipped with 32 GB of RAM and an Intel Ultra 7 processor with 16 cores, without a dedicated GPU. The training time of the Deep Q-Learning (DQL) based approaches could be further reduced by applying parallelization in the neural network training process.

4.1 Evaluation Methodologies

Although the nature of the problem is the same for all the techniques used in this study, the differences in their implementation require that each method be evaluated according to distinct criteria tailored to its specific characteristics. Each model, in fact, relies on a set of hyper-parameters to tune the learning process; these parameters change between implementations and have different meanings when applied to different approaches.

All hyper-parameter values used in each implementation are reported in Ta-

ble 4.1 and Table 4.2.

Method	α	γ	ϵ -min	ϵ -max
Baseline	0.05	0.9	0.05	0.6
Perceptive	0.05	0.9	0.05	0.6

Table 4.1: Q-Learning Training Hyper-parameters.

Method	α	γ	ϵ -min	ϵ -max	batch size	buffer size	target update
Perceptive	0.0001	0.99	0.01	0.5	64	10^6 items	20 epi.
Visual	0.0005	0.99	0.01	0.5	64	10^6 items	20 epi.

Table 4.2: Deep Q-Learning Training Hyper-parameters.

The learning rate (α) and the discount factor (γ) are used to update the Q-values in the value function (Eq. 2.1). In DQL, instead, α represents the learning rate used in the gradient descent optimization during backpropagation. The hyper-parameter ϵ controls the probability of selecting random actions, varies over time, as described in Section 3.2, between fixed minimum and maximum values. The batch size and buffer size are hyper-parameters that regulate the composition of the Experience Replay mechanism, while the target update parameter determines how frequently the target network is updated in DQL approaches.

Each agent is trained for a different number of iterations depending on the complexity of the approach and its convergence speed. The maximum number of iterations is reported in Table 4.3, but an early stopping strategy is employed in each training run to halt the process ahead of time if necessary, as described in Section 4.1.1.

Method	Episodes	Generations	Total
Baseline Q-Learning	200	500	100000
Perceptive Q-Learning	200	500	100000
Perceptive Deep Q-Learning	20	500	10000
Visual Deep Q-Learning	20	100	2000

Table 4.3: Training iterations for every experiment.

To better evaluate the performance of each agent, a set of training and validation metrics is adopted. These metrics provide an evaluation of both the learning

process and the final model performance, including the total number of effective training iterations, the overall training time, and the number of new states discovered during training.

In the case of Q-Learning (QL), the average Δ_Q (Eq. (4.1)) is computed at runtime by evaluating the magnitude of the temporal-difference update applied to the Q-values, which serves to monitor the convergence of the model.

$$\Delta_Q = \text{learning_rate} \cdot (\text{reward} + \text{discount} \cdot \text{best_future_q} - \text{prev_q}) \quad (4.1)$$

For the two DQL approaches, the training loss of the neural network is monitored and plotted. This metric is particularly useful for assessing whether the chosen network architecture fits the problem or requires further adjustments.

During training, two sets of charts are used to visually evaluate the performance of the agent. The first set shows the average distance and the average reward computed across all episodes within each generation for all the generations until completion. The second set similarly illustrates the progression of performance during a test run executed after each generation, with the exploration parameter ϵ set to zero.

For each approach, multiple experiments are conducted by tuning the hyperparameters and refining the adopted strategies in order to achieve the best possible outcomes. Intermediate results are omitted to focus on the best selected configurations.

4.1.1 Early Stopping

The training process of the agents can sometimes be unpredictable in terms of convergence due to the probabilistic elements introduced by RL and time-consuming due to the extensive exploration required in large state spaces. Because the duration of training is unpredictable, agents often reach a solution before completing the maximum number of training iterations. In such cases, an early stopping mechanism is applied when an agent satisfies a predetermined condition, causing the training process to terminate early.

```
1
2     t = 30 # Tollerance
3     distance = test['distance']
4
5     if (end_dist - t) <= distance and distance <= (end_dist + t):
6         conv_count += 1
7     else:
8         conv_count = 0
9
10    if conv_count == 5: break
11
```

Figure 4.1: Python implementation of the early stopping mechanism.

Specifically, if an agent reaches the end of the stage during a test run for more than five consecutive generations, the training is stopped and the relevant statistics are returned (see extract of code in Figure 4.1).

In contrast to the other approaches, the Visual Deep Q-Learning model employs a different early stopping mechanism by terminating training upon the first completion of a test run to prevent excessively long training times and potential memory overflow issues.

Because each agent may train for a different number of iterations (where one iteration corresponds to a complete run of the stage until victory or defeat), the total number of iterations and the training time are recorded and returned when early stopping is triggered.

4.2 Training Outcomes

The first part of this section is dedicated to comparing the best results achieved by each method. Subsequently, the training statistics of each model are visualized to provide an easier understanding of the effects of the different approaches on the problem. The final part briefly analyses the supporting elements and changes applied to the various methods in order to improve the performance of the models.

By examining the results obtained from testing the trained models in the environment (see Table 4.4), it can be observed that the best solutions were achieved using classical Q-learning approaches. These models were able to learn policies capable of completing the task by reaching the end of the level while collecting a considerable number of coins.

By analysing the behaviour of the first two agents, it is clear that the baseline agent follows a policy primarily focused on reaching the end of the level, while the perceptive agent attempts to maximize the number of collected coins in the

process. These behaviours are justified by the different reward values assigned to progress and coin collection in the two approaches, as well as the perceptive agent's ability to perceive the position of coins from afar, which is not available to the baseline agent.

When transitioning from the Q-table representation to the Deep Q-Network (DQN) approach, the perceptive agent was no longer able to learn an effective behaviour in the considered environment. The resulting policy was unable to consistently reach the end of the level, let alone collect a significant number of coins.

The Visual Deep Q-Learning model, on the other hand, was able to make progress in the environment and reach the end of the level. However, due to memory and training time limitations, the training process could not be advanced extensively, resulting in a suboptimal policy.

Method	Steps	Distance	Coins	Victory
Baseline Q-Learning	617	4230	10	Yes
Perceptive Q-Learning	627	4230	13	Yes
Perceptive Deep Q-Learning	223	1020	1	No
Visual Deep Q-Learning	489	4230	8	Yes

Table 4.4: Test results of each model after training.

The aforementioned results can be better understood by analysing the training metrics of each approach, which provide insight into the potential causes behind the failure of some models. Table 4.5 reports the training statistics of the best performing experiments, highlighting the efficiency of each method in solving the problem.

Method	Discoveries	Iterations	Time	Early-Stop
Baseline Q-Learning	32000	54600	3.56h	Yes
Perceptive Q-Learning	48000	71800	6,75h	Yes
Perceptive Deep Q-Learning	-	10000	2.56h	No
Visual Deep Q-Learning	-	1260	3.31h	Yes*

Table 4.5: Train statistics of each approach.

*Preemptive early-stopping

By calculating the average time per iteration for each approach, it can be observed that Q-learning-based methods show greater training efficiency compared

to DQL approaches. This difference is primarily due to the computational cost associated with training neural networks. In particular, for Visual Deep Q-Learning, the significant time required to train the Convolutional Neural Network (CNN), combined with the exploration of the environment, makes this approach considerably slower than its counterparts.

As previously mentioned, the Perceptive Q-learning model tends to construct a policy that prioritizes coin collection over other objectives. This tendency is also reflected in the number of states explored and the total number of iterations, which are substantially higher than those of the baseline model. In contrast, the baseline agent explores fewer states and requires fewer iterations, as it is primarily focused on reaching the end of the level rather than seeking additional objectives.

By having the same observation configuration, the Perceptive Deep Q-Learning agent exhibits a similar behaviour to seek multiple objectives. However, in this case, the DQN is unable to extract sufficient patterns from the data to properly balance the policy and converge to an optimal solution in the given training time.

By introducing a visual representation of the environment, the Visual Deep Q-Learning model is able to extract richer information from the input data, which consists of a large set of attributes. This enhanced representation enables a more effective training of the DQN and faster development of the policy.

Starting from an analysis of the progressive training metrics of the baseline approach, the model's learning appears to be improving steadily over time.

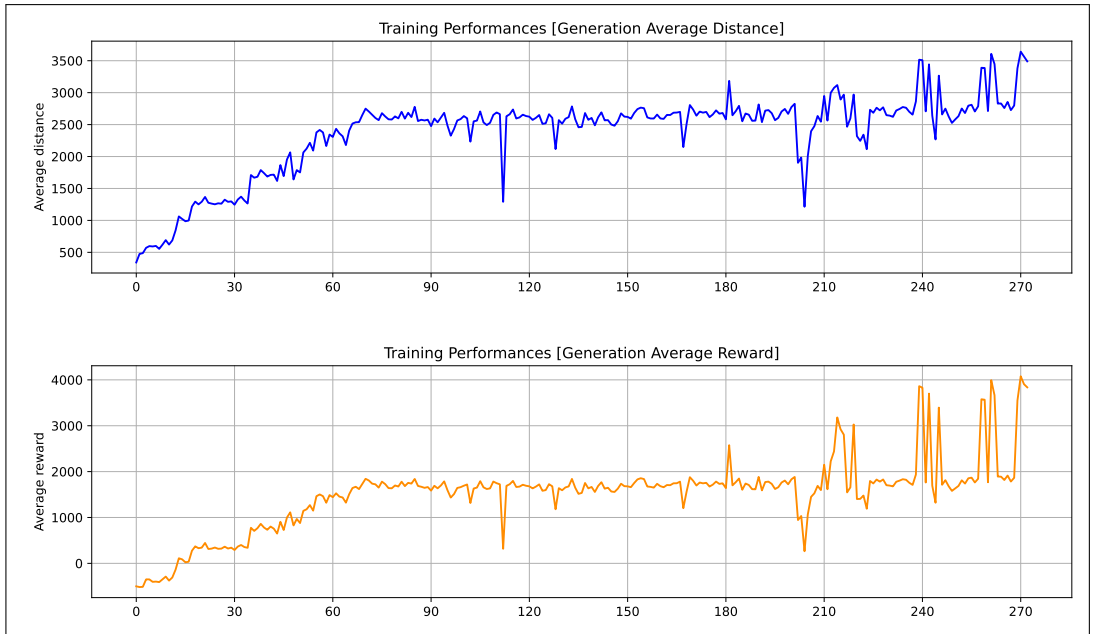


Figure 4.2: Performance and reward trend of the baseline agent during training.

As shown in Figure 4.2, the implemented exploration policy is able to keep up with the agent’s learning, maintaining the maximum distance reached during each episode stable and consistent until the agent’s decision confidence is sufficiently high to allow further progression in the level. This is further highlighted by the absence of long-term declines in the chart, indicating that previously learned states do not need to be reconsidered.

The validation metrics (Fig. 4.3), computed after each generation, are consistent with the observations made during training. Once the agent reaches a new personal best distance, it generally maintains the learned behaviour that made it possible to achieve that performance. In terms of reward, the values largely correspond to the agent’s progress in distance, with the exception of runs that end in a stalling state, where the agent does not receive a significant negative reward.

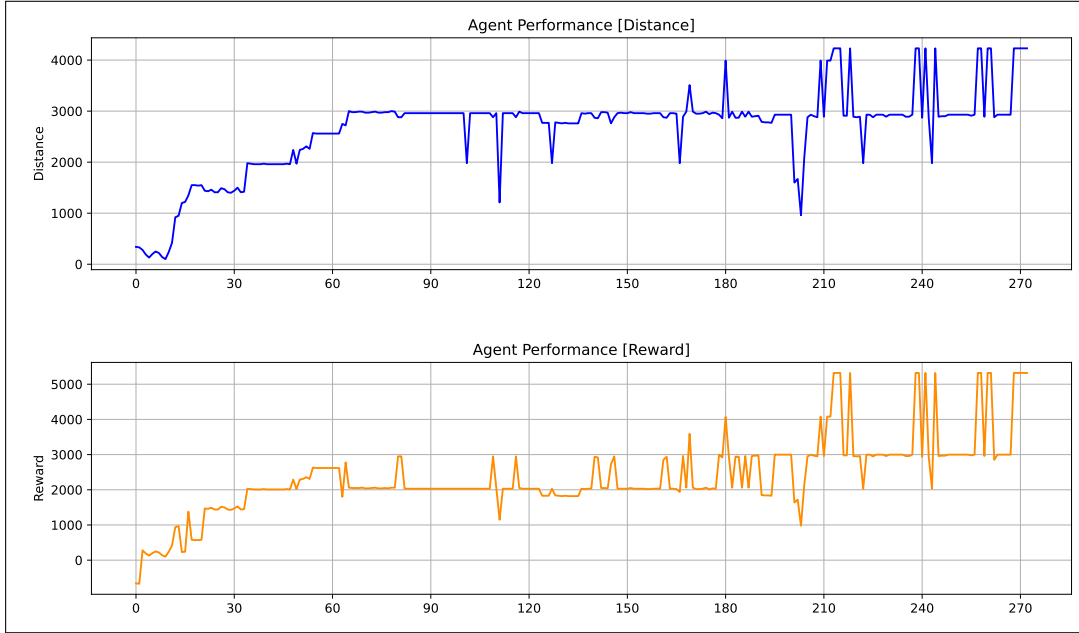
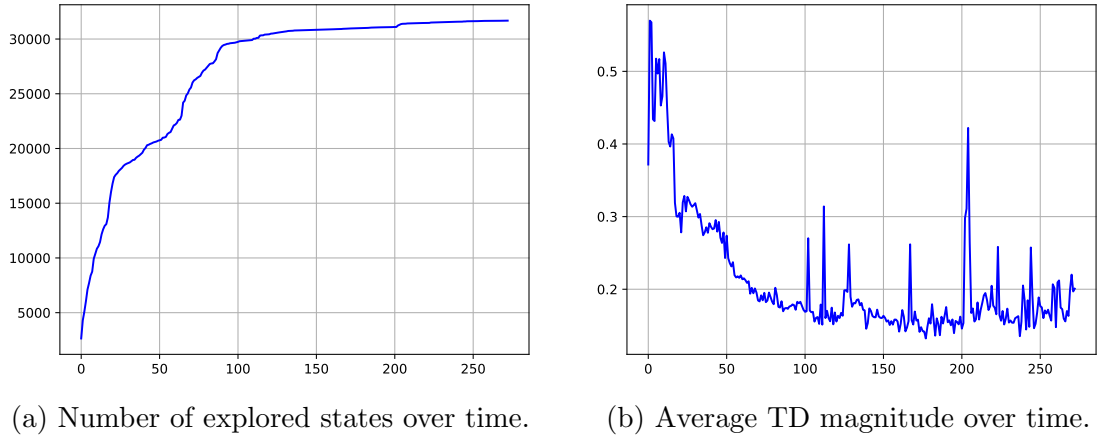


Figure 4.3: Performance and reward trend of the baseline agent in the test runs.

The number of states discovered by the agent during training increases approximately logarithmically (see Fig. 4.4a), with larger increments corresponding to generations in which a new maximum distance is reached, and smaller ones in the later stages of training. Meanwhile, the average Δ_Q (Eq. (4.1)), shown in Figure 4.4b, correctly decreases, indicating the convergence of the Q-values in the table.



As discussed in Section 3.6, the introduction of a state representation composed of multiple attributes provides the agent with more information, enabling more informed decision-making. On the other hand, the increase in information implies a corresponding increase in the time required to evaluate states and aggregate the attributes to pursue multiple objectives. By observing Figure 4.5, it is evident that the agent focuses more on gradual optimizations rather than rapid changes, leading to a more balanced and effective solution.

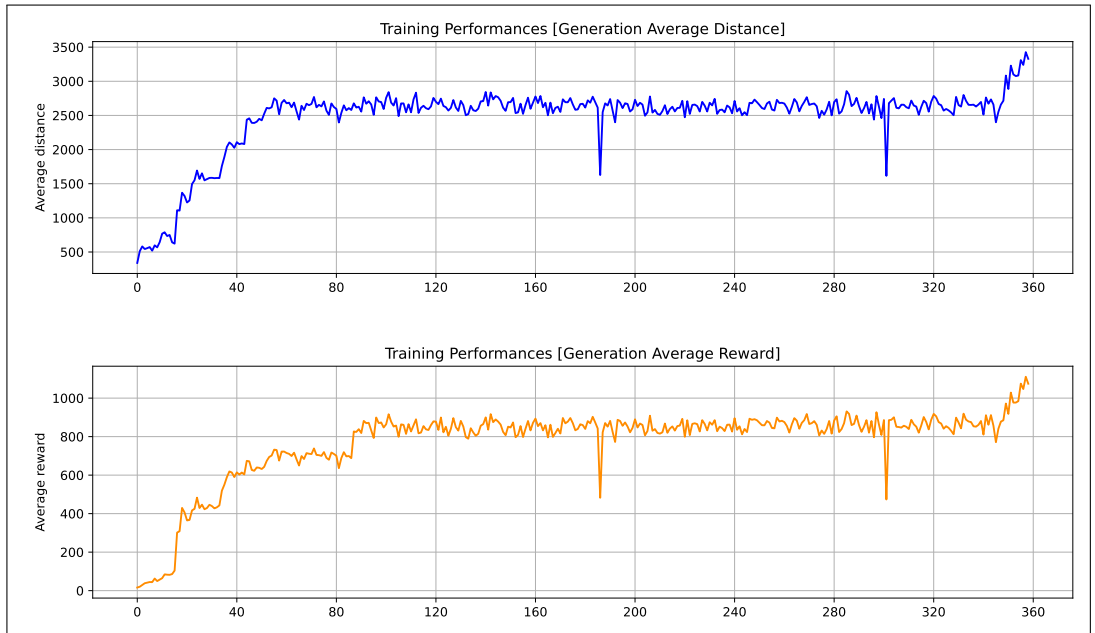


Figure 4.5: Performance and reward trend of the Perceptive Q-Learning agent during training.

In this approach, thanks to the combination of a larger observation space and a reward function that emphasizes the primary objective of the problem, the learning of the policy appears more stable and predictable than in the previous approach (Fig. 4.6).

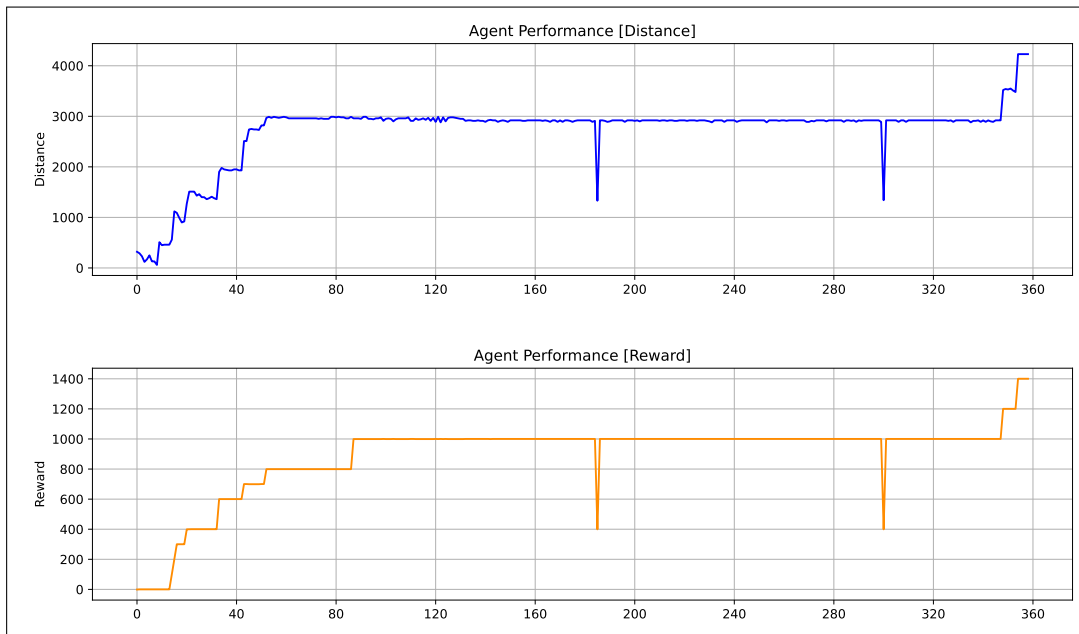
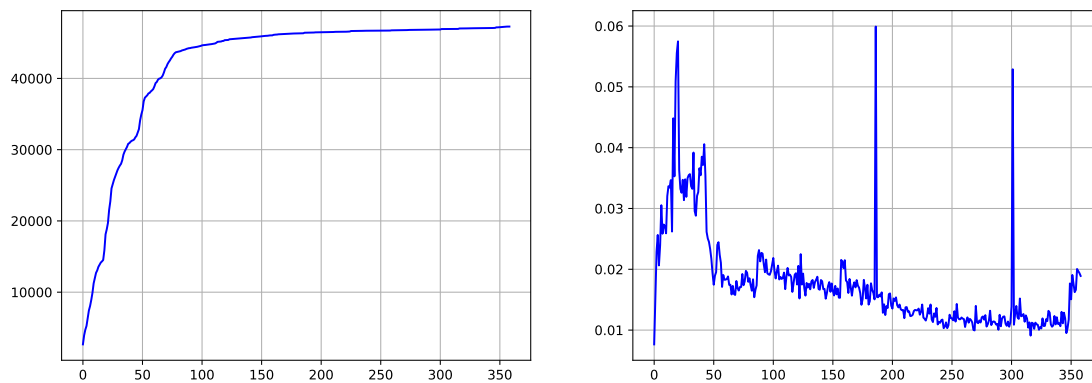


Figure 4.6: Performance and reward trend of the Perceptive Q-Learning agent in the test runs.

The plots in Figure 4.7a and Figure 4.7b resemble those of the previous approach, with the temporal difference trend during training displaying a slightly smoother curve.



(a) Number of explored states over time.

(b) Average TD magnitude over time.

From the training performance of the Perceptive Deep Q-Learning agent, the change in the learning process from the previous experiments is clearly noticeable.

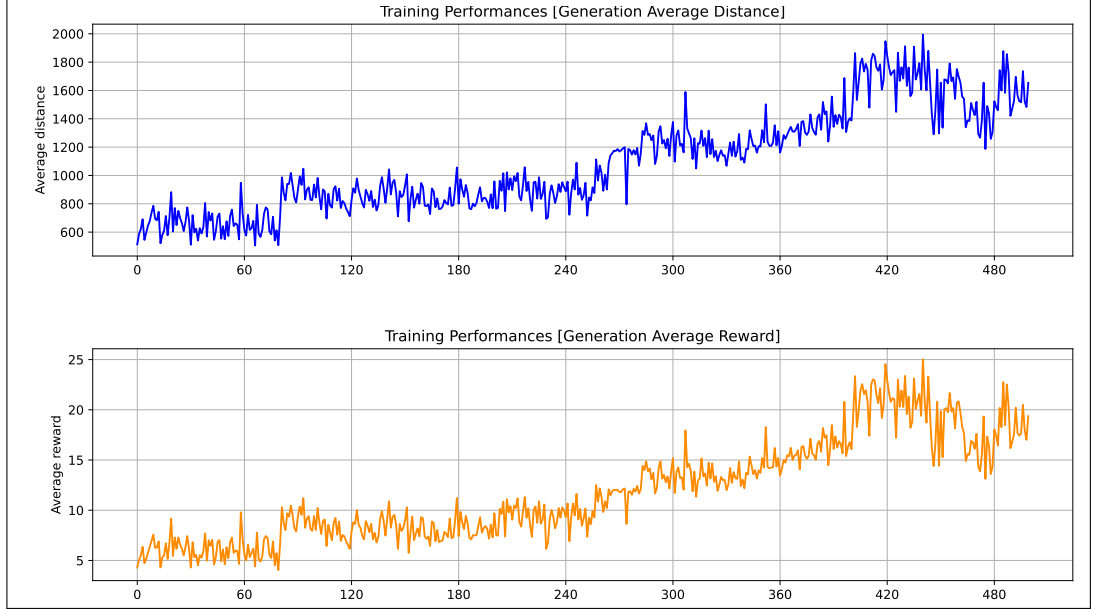


Figure 4.8: Performance and reward trend of the Perceptive Deep Q-Learning agent during training.

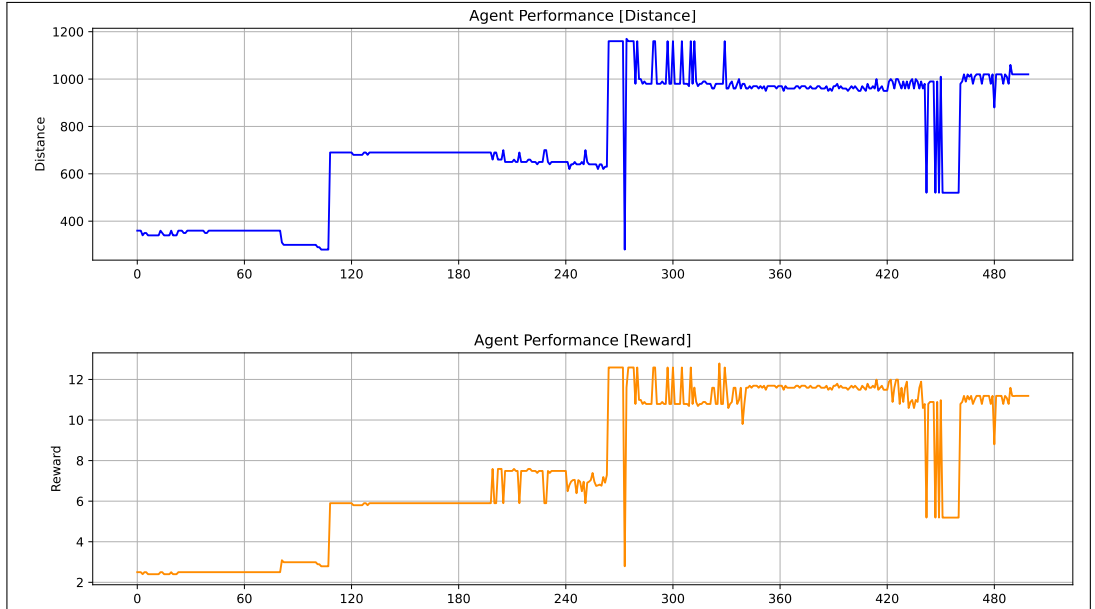


Figure 4.9: Performance and reward trend of the Perceptive Deep Q-Learning agent in the test runs.

The agent’s training appears more unstable (Fig. 4.8), primarily due to the moving target problem introduced in Section 3.7. In the long run, the network begins to struggle to retain previously learned behaviours when new states are registered, as observed in Figure 4.9.

The moving target problem in the DQN is also clearly visible in the loss plot (Fig. 4.10). At the beginning of training, the loss decreases, as the replay memory contains only similar initial states. However, as training progresses, more diverse states are discovered, and the network must adapt its learned patterns to accommodate the increasingly varied dataset. Each time the policy and target network are synchronized, the loss spikes since the Q-values predicted by the target network are updated to provide a more accurate estimation. In DQL, this behaviour is expected and indicates that the network is correctly adapting to the problem.

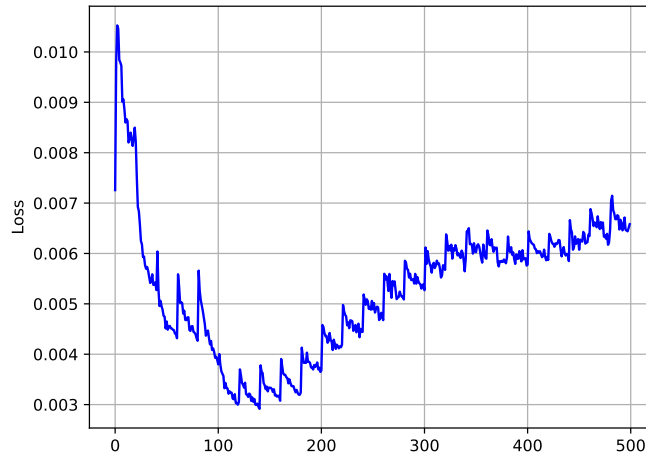


Figure 4.10: Perceptive Deep Q-Learning training loss over time.

From the training process (shown in Fig. 4.11) of the Visual Deep Q-Learning agent, it is possible to observe that learning occurs much faster, in terms of iterations, compared to the other tested approaches. The use of a wider state representation, in conjunction with a CNN, makes the extraction of meaningful patterns from the data a significantly easier task.

As explained in Section 4.1.1, due to time constraints and memory limitations, the training of the agent could not be conducted extensively. As demonstrated in Fig. 4.12, training was stopped at the first successful completion of the level, even though the average performance across the training runs of that generation was not consistently close to reaching the end of the level.

Experiments conducted beyond the first completion revealed, to a certain extent, the same behaviour observed in the Perceptive Deep Q-Learning approach, where the network does not converge to a stable solution and experiences what is



Figure 4.11: Performance and reward trend of the Visual Deep Q-Learning agent during training.



Figure 4.12: Performance and reward trend of the Visual Deep Q-Learning agent in the test runs.

known as catastrophic forgetting, a process in which previously learned features are overwritten by the introduction of newer samples, which is a common issue when dealing with large dynamic datasets.

Nevertheless, the final learned policy obtained from this experiment is able to reasonably complete the task.

The effect of the PatchLayer can be observed in its output feature map (Figure 4.13), where the 74 channels correspond to patches extracted from the three stacked input frames. Each patch captures different elements of the original images, with only a few patches displaying the most informative features.

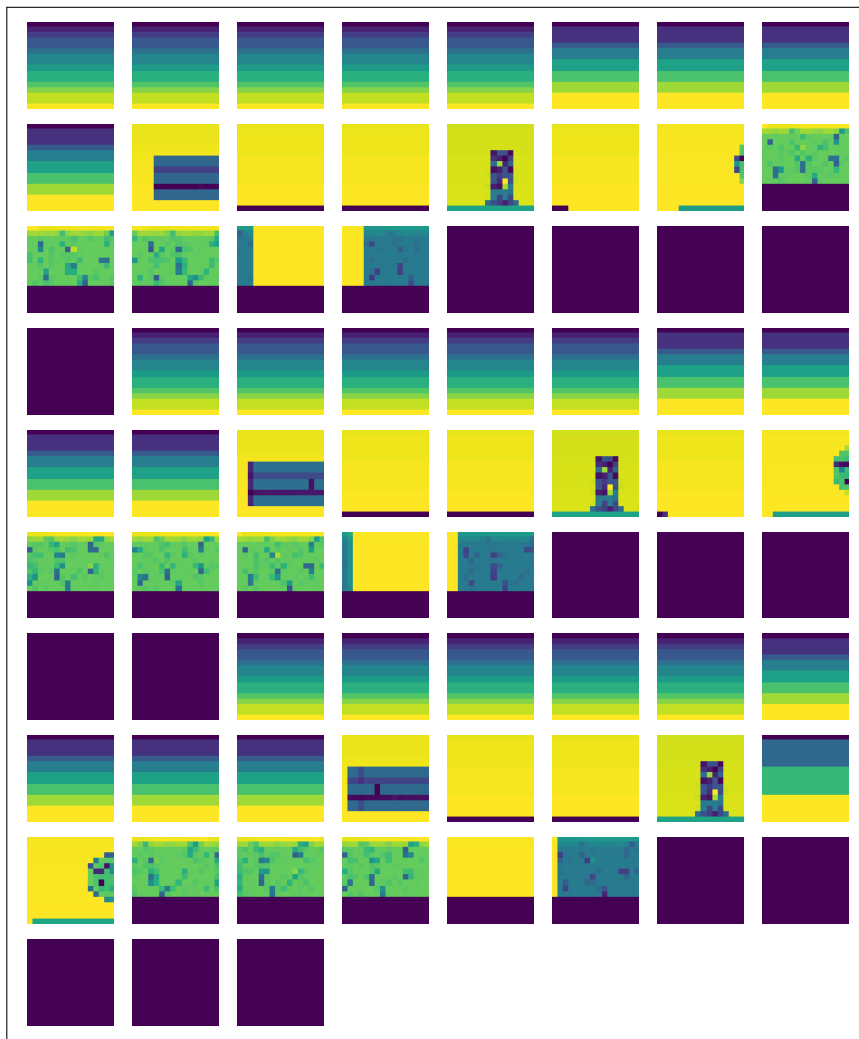


Figure 4.13: Example feature map produced by the PatchLayer of the trained network.

From all these features, the subsequent convolutional layer performs further processing, producing a more compact feature map, as shown in Figure 4.14, where it is possible to observe low-level features, such as edges and corners.

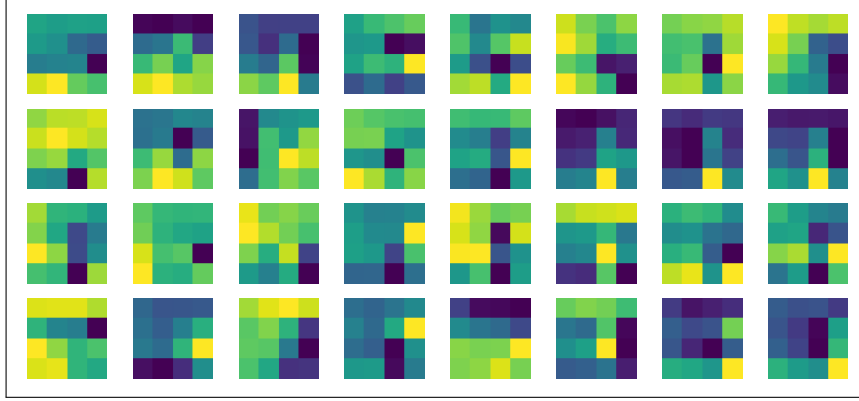


Figure 4.14: Example feature map produced by the convolutional layer of the trained network.

While evaluating the performance of the DQN, it was observed that the frequency of target network updates significantly affects training behaviour. Frequent updates of the target network lead to faster learning; however, the stability of the training is heavily compromised, and vice versa. Updating the target network every 20 episodes was found to provide a good balance between stability and convergence.

Another aspect investigated during the experiments concerns the organization and sampling strategy of the replay buffer. An attempt was made to store only unique states in order to prevent replaying repetitive, uninformative transitions. However, it was observed that the network still requires such data to learn the overall policy effectively. Furthermore, when using a very small replay buffer, the models converge rapidly to a solution determined by the limited set of samples, but they quickly forget previously acquired knowledge as older samples are replaced.

Regarding the replay buffer sampling strategy, two additional experiments were conducted in which batches of samples were balanced based on the action taken in the transition and the value of the reward obtained, either positive, negative, or zero. In both cases, modifying the replay strategy did not significantly improve the quality of the learned policy; instead, it introduced bias into the training process.

A final experiment involved dynamically adjusting the learning rate during training, decreasing it when performance improved and increasing it when performance worsened. However, in the end, this approach did not provide noticeable benefits in terms of training stability.

Conclusions

Primarily, this thesis aimed to explore the feasibility of applying Deep Reinforcement Learning (DRL), and in particular Deep Q-Learning (DQL), to large and partially observable state spaces, such as those encountered in image-sequence based problems, as well as to compare multiple approaches in order to evaluate different strategies for achieving the desired result. For this purpose, a custom video game was developed to ensure consistent evaluation across experiments and to maintain full control over the environment.

The tested Reinforcement Learning (RL) techniques demonstrated both advantages and disadvantages when applied to the studied environment, implying a non definitive solution to the problem.

Traditional Q-learning approaches based on Q-tables, although easy to implement and understand, struggle to rapidly learn appropriate behaviours when the state space grows beyond a certain size. Furthermore, their generalization to similar states or environmental changes proves to be very limited. While the tabular structure theoretically allows convergence to an optimal solution, it becomes impractical in terms of memory requirements and state representation for problems involving a large number of states or continuous learning applications.

Despite these limitations, the two tabular approaches proved to be effective in solving the presented task, maintaining a reasonable level of consistency.

When comparing the two, it can be concluded that the use of a richer observation space, supported by an appropriately designed reward function, enables better estimation of the policy, despite an increase in training time.

Regarding approaches based on DQL, training stability begins to fail, often producing inconsistent and occasionally unsatisfactory results.

De facto, when dealing with neural networks, what primarily matters is the quality and quantity of the data used for training. This means that wider observation spaces, such as raw pixel image states, and large amounts of training data enable the model to learn recurring patterns and produce more accurate estimates of the proper behaviour. Ultimately, the strength of DQL lies in its ability to extract meaningful information from high-dimensional observation spaces. However, this strength can also represent a limitation, as such representations are not always

available. In this case, when the state representation is too limited, the approach may fail, as demonstrated in the Perceptive Deep Q-Learning experiment.

It was observed that, by directly processing raw pixel frames, the Visual Deep Q-Learning agent was able to construct a policy capable of progressing within the environment. Although this method required substantial memory usage and long training times, the results were promising, highlighting the potential for further improvements, particularly with more advanced hardware, which could lead to a more optimized solution with the capability of generalizing to similar problems.

Finally, the exploration policy adopted across all implementations proved sufficient to provide adequate training samples for the models. Nevertheless, alternative exploration strategies could potentially enable a faster and more uniform state space exploration.

Future Work

Deep learning models generally tend to over-fit when trained on unbalanced or non-representative datasets. Similarly, a Deep Q-Network (DQN) agent may become biased toward accurately predicting only the most frequently encountered states. To mitigate this behaviour, a possible direction for future work would be the implementation of a mechanism that ensures more uniform exploration of the state space prior to training the model. For instance, initializing each episode from different positions within the environment could increase the diversity of visited states beforehand, producing a more stationary training dataset and potentially mitigating catastrophic forgetting.

Continuing along the direction of this thesis, further experiments could be conducted using optimized learning strategies and more suitable hardware, such as GPUs, in order to better assess the full potential of DQL applied to image based inputs, without the constraints imposed by limited computational resources.

With regard to the architecture adopted for DQN, further development could involve experimenting with alternative layer designs that are less common in conventional deep learning. In particular, the proposed PatchLayer could be refined by filtering out low-information patches or by selecting patches through a pretrained object detection mechanism to improve the saliency of the features.

Finally, an interesting and relevant future development would be to transfer an improved version of the Visual Deep Q-Learning approach to a real-world scenario. For example, a robotic agent could be trained to navigate an environment, locate and collect objects, and avoid obstacles autonomously.

Bibliography

- [1] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.
- [2] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- [3] Matthew J Page, Joanne E McKenzie, Patrick M Bossuyt, Isabelle Boutron, Tammy C Hoffmann, Cynthia D Mulrow, Larissa Shamseer, Jennifer M Tetzlaff, Elie A Akl, Sue E Brennan, Roger Chou, Julie Glanville, Jeremy M Grimshaw, Asbjørn Hróbjartsson, Manoj M Lalu, Tianjing Li, Elizabeth W Loder, Evan Mayo-Wilson, Steve McDonald, Luke A McGuinness, Lesley A Stewart, James Thomas, Andrea C Tricco, Vivian A Welch, Penny Whiting, and David Moher. The prisma 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*, 372, 2021.
- [4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin A. Riedmiller. Playing atari with deep reinforcement learning. *CoRR*, abs/1312.5602, 2013.
- [5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518, 2015.

- [6] Adil Khan, Jiang Feng, Shaohui Liu, and Ibrahim Omara. Playing a fps doom video game with deep visual reinforcement learning. *Automatic Control and Computer Sciences*, 53:214–222, 07 2019.
- [7] Ithan Moreira, Javier Rivas, Francisco Cruz, Richard Dazeley, Angel Ayala, and Bruno Fernandes. Deep reinforcement learning with interactive feedback in a human–robot environment. *Applied Sciences*, 10(16):5574, August 2020.
- [8] Jing Xin, Huan Zhao, Ding Liu, and Minqi Li. Application of deep reinforcement learning in mobile robot path planning. In *2017 Chinese Automation Congress (CAC)*, pages 7112–7116, 2017.
- [9] Advait Kulkarni, Pranav Bapat, Tanvi Kulkarni, and Rajendra Pawar. Review of reinforcement learning in chrome dino game. In *2023 7th International Conference On Computing, Communication, Control And Automation (ICCUBEA)*, pages 1–5, 2023.
- [10] Samuel Alvernaz and Julian Togelius. Autoencoder-augmented neuroevolution for visual doom playing, 2017.
- [11] Daiki Kimura. Daqn: Deep auto-encoder and q-network, 2018.
- [12] Yuezhang Li, Katia Sycara, and Rahul Iyer. Object-sensitive deep reinforcement learning, 2018.
- [13] Victor Augusto Kich, Junior Costa de Jesus, Ricardo Bedin Grando, Alisson Henrique Kolling, Gabriel Vinícius Heisler, and Rodrigo da Silva Guerra. Deep reinforcement learning using a low-dimensional observation filter for visual complex video game playing, 2022.
- [14] OpenAI, :, Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub Pachocki, Michael Petrov, Henrique P. d. O. Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski, and Susan Zhang. Dota 2 with large scale deep reinforcement learning, 2019.
- [15] Elizabeth Gilmour, Noah Plotkin, and Leslie Smith. An approach to partial observability in games: Learning to both act and observe, 2021.
- [16] Adam Tupper and Kouros Neshatian. Evaluating learned state representations for atari. In *2020 35th International Conference on Image and Vision Computing New Zealand (IVCNZ)*, pages 1–6, 2020.

- [17] Rajesh Kumar Ojha, S Janardhana Rao, Pankaj Goel, Sandeep Srivastava, K. Hareesh Kumar, and Chitturi Prasad. Cross-game generalization approaches for general video game playing using deep reinforcement learning. In *2021 2nd International Conference on Smart Electronics and Communication (ICOSEC)*, pages 1–8, 2021.
- [18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.
- [19] Jang Beakcheol, Kim Myeonghwi, Harerimana Gaspard, and Kim Jong Wook. Q-learning algorithms: A comprehensive classification and applications. *IEEE Access*, 7:133653–133667, 2019.
- [20] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning, 2015.
- [21] Michele Tomaiuolo. G2d library. <https://pkg.go.dev/github.com/tomamic/fondinfo/go/g2d>, 2022.
- [22] Ioannis Antonoglou Tom Schaul, John Quan and David Silver. Prioritized experience replay. In *ICLR*, 2016.